

Pdf

V8- μ RISC™

V8-microRISC Synthesizable 8-Bit RISC Microprocessor Core Reference Manual

Revision 9910

Disclaimer

VAutomation Reserves the right to make changes at any time, without notice, to improve design or performance and supply the best product possible. VAutomation Inc. makes no warrant for the use of its products and assumes no responsibility for any errors which may appear in this document nor does it make a commitment to update the information contained herein. VAutomation products are not authorized for use in life support devices or systems unless a specific written agreement pertaining to such use is executed between the manufacturer and an officer of VAutomation. Nothing contained herein shall be construed as a recommendation to use any product in violation of existing patents, copyrights or other rights of third parties. No license is granted by implication or otherwise under any patent, patent rights or other rights, of VAutomation Inc. All trademarks are trademarks of their respective companies.

Copyright © 1993-1999 VAutomation Inc. Nashua NH. All rights reserved.

VAutomation Inc.
402 Amherst Street
Suite 100
Nashua NH 03063
(603) 882-2282
FAX: 882-1587
<http://www.VAutomation.com>

Table of Contents

1 Installation	1
2 Release Notes	2
3 Introduction	4
4 Features	5
4.1 Reference Documents	5
5 Getting Started	6
6 Functional Description	7
6.1 Block Diagram	7
6.2 Instruction Decode (OP_DCD)	7
6.3 Arithmetic Logic Unit (V8_ALU)	7
6.4 Register File (V8_REGS)	7
6.5 Address Generation Logic (V8_ADDR)	8
6.6 DATAOUT Multiplexor	9
6.7 Programming Model	9
6.8 Opcode Table	11
6.9 Addressing Modes	12
6.10 Memory Map	12
6.11 Instruction Set Details	13
6.11.1 ADD - ADD	13
6.11.2 ADC - Add with Carry	13
6.11.3 AND - Logical AND	14
6.11.4 BR0 - Branch if 0	14
6.11.5 BR1 - Branch if 1	15
6.11.6 BTT - Bit Test	15
6.11.7 CLP - Clear PSR bit	16
6.11.8 CMP - Compare	16
6.11.9 DEC - Decrement	17
6.11.10 INC - Increment	17
6.11.11 INT - Interrupt	18
6.11.12 JMP - Jump Absolute	19
6.11.13 JSR - Jump to Subroutine	19
6.11.14 LDA - Load Absolute	20
6.11.15 LDI - Load Immediate	20
6.11.16 LDO - Load Indexed with Offset	21
6.11.17 LDX - Load Indexed	21
6.11.18 OR - Logical OR	22
6.11.19 POP - POP from Stack	22
6.11.20 PSH - Push to Stack	22
6.11.21 ROL - Rotate Left	23
6.11.22 ROR - Rotate Right	23
6.11.23 RTI - Return from Interrupt	24
6.11.24 RTS - Return from Subroutine	25
6.11.25 SBC - Subtract with Carry	26
6.11.26 STA - Store Absolute	26
6.11.27 STO - Store Indexed with Offset	27
6.11.28 STP - Set PSR bit	27
6.11.29 STX - Store Indexed	28
6.11.30 T0X - Transfer R0 to Rn	28

6.11.31 TX0 - Transfer Rn to R0	28
6.11.32 UPP - Increment Register Pair	29
6.11.33 XOR - Logical Exclusive-OR	29
6.11.34 XSP - Exchange Stack Pointer	30
7 Signal Description	31
7.1 Address Bus (ADDR[15:0])	31
7.2 Data Bus (DATAIN[7:0], DATAOUT[7:0])	31
7.3 Write Enable (WRITE, WRITE_D)	32
7.4 Read Enable (READ)	32
7.5 RESET	32
7.6 Interrupts (INT[7:0])	32
7.8 Set/Clear PSR Bits (SET_P, CLR_P)	33
7.9 Stack Pointer Overflow (SP_OFLO)	33
7.10 Opcode Fetch (OP_FETCH)	33
7.11 Processor Status Register (PSR)	33
8 Timing	33
9 V8 Standard Peripherals	35
9.1 ASIC_V8	36
9.2 VUART 1000 gates	37
9.3 VTIMER 1600 gates	39
9.4 V1284	41
9.5 VPIO (400 gates)	42
9.6 PWR_SAVE Unit 400 Gates	43
9.7 VSPI Serial Peripheral Interface (800 Gates)	45
9.8 V8_DEICE	48
9.8.1 BASIC architecture features (3500 gates)	48
10 Synthesizing the V8-uRISC	51
10.1 Makefile	51
10.2 V8-uRISC Microcontroller File Hierarchy	51
11 Test Bench Description	52
11.1 V8-uRISC Test Bench File Hierarchy	53
11.2 Executing the Test Bench	54
11.3 Notes for Verilog Users	55
11.4 Assembly language software	55
12 V8 Software Development Tools	56
13 Tips and Techniques	56
13.1 Debugging V8 software in an HDL simulator	56
13.2 Interfacing to Off-Chip Memories	56
13.3 Expanding the Address Space Beyond 64K bytes	56
14 IntelliCore™ Prototype System	57

1 Installation

Typically the V8-microRISC core is shipped via email, a single CD or on a single DOS compatible 3.5 inch floppy disk. The following description details how to install the V8 core onto a PC. If you have requested shipment on other media, follow the instructions on the label of the media shipped. You will need the Authorization Key from your signed license agreement to install the core.

If you have any problems or question installing the core, please call our hotline at (603) 882-2282 or via Internet Email at support@VAutomation.com.

To install VAutomation products, run Setup.exe from the attachment to the email or on the floppy.

Setup will ask you where to put the VAutomation files.

Setup will recommend C:\vhdfiles but you may pick another directory and/or another drive. You may even specify a network drive such as \\Server\user_name\vhdfiles.

Once you have specified where the files are to be installed, click "Next".

You may then view our license agreement, and signify your acceptance of it by clicking "Agree".

Then enter your Authorization Key and click "Next".

Then enter your name and click "Next".

Then enter your company name and click "Next".

Then enter your site name and click "Finish".

All VAutomation files will be copied to the specified directory. There are no hidden files. No files are copied to any other directory.

If you wish to remove all the VAutomation files, run `uninstall.exe`.

A Unix "make" utility file is shipped with the V8. This utility provides command line examples on how to build a simulation database and synthesize the V8. Depending on your Computer Aided Engineering (CAE) tools, you may need to setup various environment variables and your \$PATH to properly use the Makefile commands. See the comments in the Makefile file for more information. If you are using PC based tools, we highly recommend the MKS toolkit which will provide the "make" utilities required by the supplied makefile. MKS can be contacted at [HTTP://www.mks.com](http://www.mks.com)

Type "make sim" to create a simulation database.

Type "Make syn" to synthesize the V8 model.

2 Release Notes

September 21, 1999: Rev 9910 Release

BUG Fixes: The V8_DEICE.VHD file was changed to fix a minor bug which prevented the DeICE JTAG debugger from properly capturing histogram data. When a program was composed almost exclusively with single clock opcodes, then the histogram data was not captured. Note that this does not affect normal debugging operation, just the capturing of histogram data.

ENHANCEMENTS: Many files have been changed and signal names and entities have also changed to make all V8 files as compatible with the industry coding standard Reuse and Methodology Manual (RMM). The VHDL libraries were also changed from STD_LOGIC_ARITH (a Synopsys library) to NUMERIC_STD (an IEEE standard). These changes will make VAutomation cores more familiar and easier to interface to other IP blocks available from other IP vendors.

The empty VUSB and VUSB-HUB modules have been removed from the V8 release which makes the ASIC_V8 design much simpler and easier to understand for non-USB users.

The SP_OFLO signal has been added to provide an indicator when the stack pointer has overflowed.

April 2, 1999: Rev 9904 Release

BUG Fixes: The Stack Pointer (SP) would write to address 0x2FF instead of 0x3FF when an 8-bit SP wrapped from 0x300->0x3FF. The V8_ADDR.VHD was fixed to correct this problem.

The VUART was testing the wrong bit of the STATUS register to gate the RX interrupt bit. The VUART.VHD file has been updated to use the correct bit as described in this document.

ENHANCEMENTS: The RSP opcode has been replaced with the XSP opcode to provide RTOS support. The RSP opcode would simply reset the stack pointer to its power up value. It was originally added to provide full ANSI C compliance and the C compiler will only use RSP to support one obscure ANSI C error routine. The XSP opcode provides complete control of the stack pointer by exchanging R0/R1 with the stack pointer.

The SPI core has been changed to be fully synchronous with the V8 instead of having some of the logic operate on the SPI SCK clock signal. This provides a much easier to use core as there is a single clock instead of multiple clocks.

The VJTAG module is now fully synchronous with the CPU clock. Previously the VJTAG module clocked on the JTAG TCK signal. TCK is now sampled and all operations take place synchronously with the CPU clock. This removes the complications of additional clock domains and the asynchronous flip-flops originally required.

The V8_DEICE JTAG debugger was improved for synchronous reset with Synopsys and to be compatible with the new synchronous VJTAG module.

The PWR_SAVE module was changed to bring the three asynchronous reset flip-flops out of the PWR_SAVE module and up to a higher level in the hierarchy. This was done to provide closer control of these cells rather than having the logic deeply buried in the V8. Typically you will want to instantiate these Flip-Flops and specify their placement. The routing of clock signals is also simplified as the four power save control signals are driven up to the top level of the hierarchy rather than having to run the clocks down to the PWR_SAVE module and then back up again.

November 20, 1998: Rev 9811 Release

BUG FIX: A few minor bug fixes were made to V8_DEICE, VSPI and PWR_SAVE.

ENHANCEMENTS: The WRITE_D signal was added to the V8 and to the V8_DEICE core to ease the timing requirements for an external WE_N signal. This significantly improves the hold time on WE_N.

September 30, 1998: Rev 9809 Release

BUG FIX: The 3 undefined opcodes in the JMP group previously would have resulted in the V8 hanging and never fetching another opcode. These opcodes now function as 5 clock NOPs. Note that the 0xBB opcode is now reserved for use by the JTAG debugger.

ENHANCEMENTS: Updated the VTIMER module with a completely new design. Added the VSPI peripheral. The VUART module has been updated for Synopsys and interrupt capabilities were added. The V8_DEICE has been fixed when setting breakpoints. The Synopsys synchronous reset attributes and coding styles were also improved in several files. The assembler, simulator, JTAG debugger and C compiler have all been improved. Documentation for the Assembler and other software development tools was moved to the V8-microRISC Software Development Tools document.

July 2, 1998: Rev 9806 release

Added the PWR_SAVE peripheral which implements four "sleep" modes which reduce power consumption of the V8. The testbench has been changed from the older V8_SBC and VLM PCB example to the newer IPS_PCB format. This version of the IPS provides support for Altera 10KA FPGAs as well as Xilinx XC4000XL devices. Other minor improvements have been made to the assembler and the V8_DEICE JTAG debugger. The VPC_TERM.DAT file is now converted to a HEX file which can be read into Verilog and thus support for it is now available.

April 23, 1998: Rev 9804 release

This release makes a number of improvements to the V8. The performance has been improved by adding a pipeline stage to the address bus. The XRM16X8 module was integrated into the V8_REGS module eliminating a redundant level of hierarchy. Verilog support has been significantly improved however the VPC_TERM module does not yet read in the VPC_TERM.DAT file and drive the characters into the V8 UART due to the fileio limitations of Verilog. The assembler has been significantly improved with the addition of conditional assembly directives and macros.

November 7, 1997: Rev 1.0 release

3 Introduction

The V8-microRISC™ 8-bit RISC microprocessor is a general purpose, synthesizable processor core designed specifically to be embedded in both ASICs and FPGAs. The V8-microRISC™ and its available peripherals allow true system-on-chip integration using any ASIC or FPGA technology. Supplied with a comprehensive set of support tools, the V8-microRISC™ will reduce your development cost and risk while speeding your product to market.

High Performance and Low Power

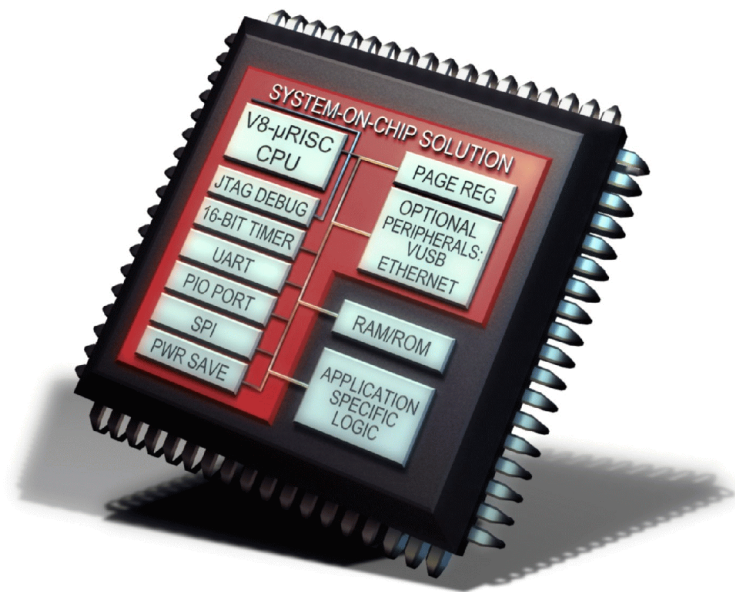
The V8-uRISC™ combines a small gate count with single clock cycle execution for many instructions to deliver a high performance 8-bit microprocessor with a very small footprint. The V8-uRISC™ is ideal for 8-bit applications which need a combination of small size, good performance and low power.

Small Size Yields SOC Integration

Although the V8-uRISC™'s small size is an enabler for system-on-chip (SOC) integration using any ASIC or FPGA technology, it is especially well suited for FPGA applications. The V8-uRISC™ along with several peripherals can be implemented in an FPGA priced in volume under \$5. The result is a low-cost microcontroller customized for your application.

Robust Set of Peripherals

The V8-uRISC™ is delivered with a set of standard peripherals including a 16-bit Counter/Timer, UART, Power-Save Unit, SPI, I2C EEPROM interface, P1284 parallel interface, and page register design to expand the address space to 8 megabytes. Optional Universal Serial Bus Device, HUB or Host interfaces are also available.



Technology Independent and Customizable

Delivered in either VHDL or Verilog synthesizable source form, the V8-uRISC™ is both technology independent and user-customizable. Your ASIC or FPGA synthesis tool can target the V8-uRISC™ to the process/library of your choice. During this targeting step the synthesis tool can perform architecture specific optimizations that would be impossible to do if only a netlist were supplied. Additionally, the availability of source code allows custom opcodes to be added for improved throughput at a lower clock frequency.

Many of today's RISC processors use a Harvard¹ CPU architecture to increase performance. But this increase in performance comes at a cost of many more gates, more wire routing and is less flexible in RAM configuration. The V8 uses a traditional Von-Neuman² CPU architecture in order to minimize the

¹A Harvard CPU Architecture is characterized by having separate datapaths (and typically memories) for the processors executable code and data. Having separate datapaths yields higher performance as opcodes and data can be fetched from memory in a single cycle.

² A Von Neuman CPU Architecture is characterized by having the processors executable code and the data to reside in the same physical area in memory.

gate count and provide the maximum flexibility in RAM configurations. Typically the V8 is used in a low-cost, low-power device where RAM is at a premium. Configurations of 4K bytes or less of combined program and data space are common.

Typically the V8 is integrated with a serial EEPROM controller that will load the program code from the EEPROM at powerup. Putting the executable code in EEPROM instead of a more traditional masked ROM allows field upgrades anytime through simple downloading of the new code. It also alleviates the purchasing lead times and inventory problems associated with masked or One-Time-Programmable (OTP) microcontrollers.

The V8 is unique in that it includes two user-definable opcodes (USR,USR2). Replacing the most common, repetitive function in your application code with a single opcode can yield a dramatic increase in performance without increasing the clock frequency. Higher throughput at a lower clock frequency can significantly cut power consumption. Typical uses for the USR opcodes include operations such as block-move, 1024 bit shift/add needed for encryption, DSP pipeline functions, DECrement and JMP if zero, MUL and DIV, MUL-Accumulate, Trigonometric functions. If you need help in implementing your desired opcode, VAutomation can provide consulting services to quickly and accurately implement your opcode.

4 Features

- ! Under 2,000 Gates for the base CPU (not including the register file)
- ! Ideal processor for cost sensitive embedded applications
 - Easily fits in a corner of many FPGAs
- ! RISC architecture for low gate count and high performance
 - Single Cycle instruction execution for all register to register opcodes
 - 16Mhz operation in many FPGA families
 - 80+Mhz operation in 0.35um CMOS Area Optimized
- ! High Code density
 - Many opcodes are a single byte
- ! ASIC/FPGA Vendor Independent, Technology Independent
- ! Available in either VHDL or Verilog
- ! 34 opcodes, 4 addressing modes, 2 user defined opcodes
 - Custom opcodes for 1024 bit math, table lookups, DSP functions are easily added
- ! Von Neuman CPU architecture
- ! 8-bit ALU
- ! 64K byte addressing capability
- ! Eight 8-bit General Purpose Registers
 - Multiple register banks are easily implemented to minimize interrupt latency
- ! 16-bit Program Counter and Stack Pointer
- ! Seven maskable interrupts, one non-maskable
- ! Fully synchronous and static design
- ! No internal tri-state busses
- ! Customizable Assembler in ANSI C source code included
- ! 128 BIST ROM assembly source code
- ! JTAG Debugger available
- ! IntelliCore™ Prototyping System available for evaluation and rapid product development
- ! C Compiler available from Hi-Tech Software [HTTP://www.htsoft.com](http://www.htsoft.com)
- ! Real-Time-Operating-System (RTOS) available

4.1 Reference Documents

The following documents will provide additional details on the V8 software tools and the Prototyping System. Both are available from VAutomation and are included in the V8 release.

V8-uRISC Software Development Tools - V8tools.pdf

Intellcore™ Prototyping System (IPS) Reference Manual - ips.pdf

5 Getting Started

There are two different ways to begin integrating the V8-uRISC core into your ASIC or FPGA design. The first scenario typically begins by instantiating the V8_TOP.VHD (or V8_TOP.V for Verilog users) into your ASIC and adding other peripherals as needed. The simulation environment and test bench will be of your own design. To save time, we recommend that you look through the test bench, assembly code, peripherals and simulation models and reuse these items where appropriate. This approach is best suited to applications which are significantly different than those provided in the sample simulation environment provided.

The second scenario for integrating the V8-uRISC into your ASIC is to use VAutomation's test bench and add your logic within the provided simulation environment. In this case you will want to start with the XIL240HQ.VHD or A10K240.VHD modules which simply create the IO cells used in the FPGA implementations of the V8 for VAutomations IntelliCore Prototype System (IPS). Simply recode the IOs one of these module to match the IOs of your ASIC. Then add your own logic to the ASIC_V8.VHD file and add, remove or customize the other peripherals provided. This method will save you time since a fairly complete simulation environment has already been provided by VAutomation.

Whichever method you choose, the main point here is that you have purchased much more than just a synthesizable microprocessor. Numerous synthesizable peripherals and behavioral simulation models have been provided as well. A complete board level simulation environment which can easily be expanded to multiple board or system simulations has been provided. The environment provided is a fairly complete simulation of the IntelliCore Prototype System which is available from VAutomation for full emulation of your ASIC in FPGAs.

The only configuration parameters in the V8 are related to the location of the stack pointer and the interrupt vectors in memory. See the V8_ADDR.VHD file for details on the INTERRUPT_VECTOR and STACK_INIT parameters. Also note that at the bottom of this file are several examples on how to change the size of the stack pointer. Normally it is 8 bits or 256 bytes which is more than enough for most applications.

Be sure to check out VAutomations web page on the V8 at [HTTP://www.vautomation.com/v8/v8.html](http://www.vautomation.com/v8/v8.html) for any recent release news on the V8. The FAQ page also includes some helpful hints on debugging a microprocessor based system. The FREE page has a ZIP file with the latest version of the V8 assembler and instruction set simulator.

6 Functional Description

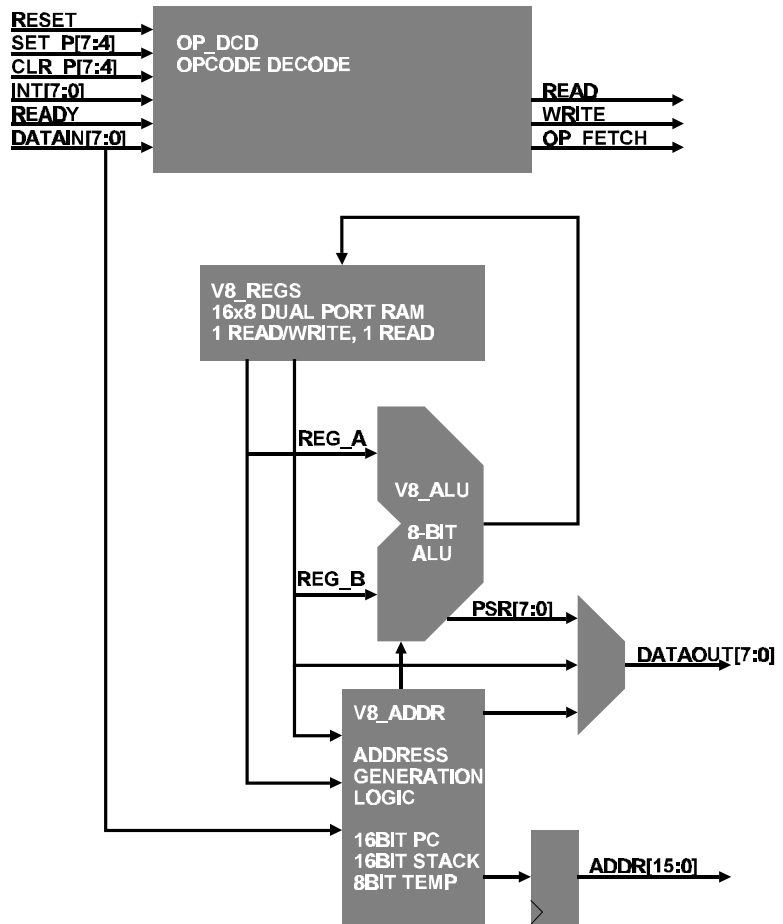
The functional blocks that make up the V8 are described below. This information is provided to help you understand the internal workings of the V8. This level of detail is helpful when you want to add your own user-defined opcodes the V8.

6.1 Block Diagram

The V8-uRISC is built upon four major blocks of logic; the opcode decode, the ALU, the register file and the address generation blocks. Each of these blocks are briefly described below. For more detail, see the HDL source code.

6.2 Instruction Decode (OP_DCD)

The Instruction Decode block controls the operation of the V8. Opcodes are fetched from memory and loaded into the Opcode Register. The opcode is then decoded and the appropriate control signals are output. The control signals depend on the opcode and the number of clock cycles that have been executed for the particular opcode. Most of the control outputs are decodes and thus may glitch and may have a significant amount of delay before settling to their final state. However, WRITE comes directly from a Flip-Flop to insure there are no glitches. The READY signal will hold the current state of the CPU as well as deactivating many of the register load control signals. The INT and RESET lines force an INT opcode to be loaded into the opcode register and special processing is performed depending on the type of interrupt.



6.3 Arithmetic Logic Unit (V8_ALU)

The ALU block performs the 8 bit arithmetic and logical operations. The inputs to the ALU come directly from the Register File or from the TEMP register in the V8_ADDR module. Certain constants such as 0 and 1 can also be selected as inputs to the ALU for increment and decrement operations. The ALU includes bit-manipulation logic to allow test, setting and clearing of a single bit of a byte. This is a common function in embedded microcontrollers. The Processor Status Register (PSR) is also located within the V8_ALU block. Most opcodes pass data through the ALU so that the appropriate PSR flags can be updated. See section 5.8 for more information on the PSR. A logic one on the SET_P signals will cause the corresponding bit in the PSR register to be set and a logic one on the CLR_P pin will clear the corresponding PSR bit to zero. These signals are typically connected to IO device status lines which allows the V8 to test and branch on these bits quickly.

6.4 Register File (V8_REGS)

The Register File is an eight byte, dual read, single write register file. The register file is typically implemented as a dual port RAM with one port being read/write and the other port being read only. Alternatively, the register file can be synthesized to D type Flip-flops and multiplexors. The registers are referred to as R0 through R7. R0 can be thought of as the accumulator as it is the default

source/destination for most opcodes. Typically R4 through R7 are used as index registers into RAM although they can be used for any purpose.

To improve interrupt performance, the register file is typically doubled in size and the 1 bit of the PSR is used to select a second bank of registers which are used only during interrupt processing. The second bank of registers allows the interrupt to be serviced without having to save and restore all of the registers. This can save several microseconds for each interrupt. Additional banks of registers can be added to allow rapid changes in context or for any purpose where the current state of the CPU needs to be retained. Typically the bank select bits are sourced from the FLAG bits in the PSR.

The V8_REGS.VHD file contains two architectures in VHDL. The SYNTH architecture will create a 16x8 register file out of D type Flip-Flops and multiplexers (about 1700 gates worth). The XILINX architecture creates a 16x8 dual port register file using the RAMD Xilinx primitive cell. This results in a tremendous savings when targeting Xilinx FPGAs (8 CLBs instead of over 64). Similar savings can also be obtained in ASIC architectures by using an ASIC cell with similar characteristics to the Xilinx RAMD cell. There is a behavioral model of the RAMD cell in the XIL_IO.VHD file. Verilog users will note that the two architectures are provided in two different files, V8_REGS.V and V8_REGS_XILINX.V.

6.5 Address Generation Logic (V8_ADDR)

The address generation logic module consists of the 16 bit program counter, the stack pointer and an 8 bit temporary register. The output of the V8_ALU block is actually the address for the next clock (ADDR_D). The ADDR_D bus is then registered external to the V8_ALU block so that the address can come directly from a Flip-Flop. The performance limiting paths in a V8 microcontroller design are typically from the ADDR bus, through address decodes to external IO, RAM or ROM and then into the DATAIN bus of the V8. Providing the ADDR bus directly from Flip-Flops results saves time in this critical path. Critical address decodes can also be done a cycle early by decoding the ADDR_D bus and then registering the address decode signal.

The 16 bit Program Counter (PC) provides the address from which opcodes are fetched. The counter is incremented whenever an opcode is fetched or the data bytes of an opcode are read. The counter is reloaded whenever a Jump or Branch opcode is executed or when processing an interrupt.

The stack pointer (SP) is a 16 bit up/down counter. It provides the address when data is pushed or popped from the stack. The SP points to the byte that was most recently pushed onto the stack. Typically the upper 8 bits of the SP are set to 0x03 but they can easily be set to any desired location in RAM. The counter can be reduced to less than 16 bits by simply changing a few constants in the HDL code to reduce the gate count of the V8 CPU. By default, the SP is set to 8 bits with the upper 8 bits being a constant 0x03. Applications that anticipate using a Real-Time Operating Systems (RTOS) may want to utilize a full 16 bit stack pointer. Generally, most applications can function with an eight or even a 5 bit stack pointer. The C compiler does not use the hardware stack pointer to pass parameters. The stack is primarily used for return addresses when calling subroutines and for interrupts. A five bit stack pointer will allow for up to fifteen levels of nested subroutines. Interrupts are more difficult to compute the stack requirements in advance since the C compiler may push the registers on the stack. However, generally the V8 does not nest interrupts and if you simply assume one interrupt which worst case will consume eleven bytes (PC, PSR and all 8 registers) this reduces the subroutine nesting to about nine with a five bit stack pointer.

The location of the stack page is set with the STACK_INIT constant. This 16-bit constant must be assigned to the starting location of the stack. Typically it is assigned the value of 0x03FF with an 8 bit stack pointer. This places the stack in the range from 0x0300-0x03FF. There are several examples at the bottom of the file which show how to change the size of the stack.

The location of the interrupt vectors is set with the INTERRUPT_VECTOR constant. Typically the interrupt vectors are placed just above the stack page (0x0400).

6.6 DATAOUT Multiplexor

The Data Out Mux is an 8-bit 3 to 1 mux. It routes data from the register file or the high and low halves of the PC. The PC is required to be output when executing an interrupt or jumping to a subroutine.

6.7 Programming Model

The V8 provides eight 8-bit general purpose registers. Most opcodes utilize register R0 as a source operand and the destination for instruction. The REG field of the opcode specifies the other operand register. The registers can be used as pairs to form 16 bit index addresses into memory.

7	6	5	4	3	2	1	0	
R0								Accumulator
R1								
R2								General Purpose Registers
R3								
R4								
R5								Register Pairs form 16-bit addresses for indexed and offset addressing modes
R6								
R7								

The stack pointer and program counter are both 16-bits. The reset value and the number of bits of the SP can be specified by constants in the HDL code. The Stack Pointer default is an 8-bit SP located in the 0x300-0x3FF address space. The **PC is always reset to zero.**

15	8	7	0
Stack Pointer (SP)			
Program Counter (PC)			

The Processor Status Register (PSR) maintains the state of the flags after most computations. The **upper four bits are connected to external signals** for rapid branching on IO signals.

7	6	5	4	3	2	1	0	
P7	P6	P5	P4	I	N	C	Z	Processor Status Register (PSR)

PSR[7] = P7 = FLAG bit, Set by SET_P[7], Cleared by CLR_P[7]
PSR[6] = P6 = FLAG bit, Set by SET_P[6], Cleared by CLR_P[6]
PSR[5] = P5 = FLAG bit, Set by SET_P[5], Cleared by CLR_P[5]
PSR[4] = P4 = FLAG bit, Set by SET_P[4], Cleared by CLR_P[4]
PSR[3] = I = Interrupt MASK bit (1=ignore interrupts, 0=enable interrupts)
PSR[2] = N = Negative bit (1 when ALU result has bit 7=1)
PSR[1] = C = Carry bit (NOT borrow for subtract)
PSR[0] = Z = Zero bit (1 when ALU result is zero)

The Opcode Format is defined as a single 8-bit opcode optionally followed by one or two additional bytes. The upper 5 bits of the opcode byte specify the operation to be performed. The low 3 bits typically provide the source register for an operand. The other operand is typically R0. Some opcodes have an additional one or two byte operand following the opcode.

7	6	5	4	3	2	1	0	Byte1	Byte2
OPCODE					REG				
OPCODE					REG			DATA	
OPCODE					REG			DATA LOW	DATA HIGH

6.8 Opcode Table

Opcode in Hex	Assembler Notation	Operand	Clock cycles	PSR	Description
00	INC	Rn	1	NCZ	Rn=Rn+1
08	ADC	Rn	1	NCZ	R0=R0+Rn+C
10	TX0	Rn	1	NZ	R0=Rn
18	OR	Rn	1	NZ	R0=R0 Rn
20	AND	Rn	1	NZ	R0=R0&Rn
28	XOR	Rn	1	NZ	R0=R0^Rn
30	ROL	Rn	1	NZC	Rn={Rn<<1,C}
38	ROR	Rn	1	NZC	Rn={C,Rn>>1}
40	DEC	Rn	1	NCZ	Rn=Rn-1
48	SBC	Rn	1	NZC	R0=R0-Rn-C
50	ADD	Rn	1	NZC	R0=R0+Rn
58	STP	#	1	*	PSR[#]=1, Set bit in PSR
60	BTT	#	1	NZ	Bit Test; if R0[#]=0, Set Z else clear Z
68	CLP	#	1	*	PSR[#]=0, Clear bit in PSR
70	T0X	Rn	1	NZ	Rn=R0, Transfer
78	CMP	Rn	1	NZC	R0-Rn, Compare
80	PSH	Rn	3		Push Rn on STACK
88	POP	Rn	3		Pop STACK to Rn
90	BR0	#,offset	3		Relative branch if PSR[#]=0
98	BR1	#,offset	3		Relative branch if PSR[#]=1
A0	USR				User Defined Opcode (NOP by default)
A8	INT	#	7	I=1	Interrupt
B0	USR2				User Defined Opcode (NOP by default)
B8	XSP		5		Exchange Stack Pointer
B9	RTS		5		Return from Subroutine
BA	RTI		5	stack	Return from Interrupt
BB	BRK		5		Reserved for debugging. Performs a 5 clock NOP
BC	JMP	16 bit addr	3		Jump to addr
BF	JSR	16 bit addr	5		Jump to Subroutine addr
C0	UPP	Rn	2	C	Increment Register Pair
C8	STA	Rn,addr	4		Store Rn to addr
D0	STX	Rn	3		Store R0 indexed MEM{Rn+1,Rn}=R0
D8	STO	Rn, offset	4		Store R0 Indexed with offset MEM{Rn+1,Rn}+offset=R0
E0	LDI	Rn, value	2	NZ	Load Rn Immediate
E8	LDA	Rn,addr	4	NZ	Load Rn from addr
F0	LDX	Rn	3	NZ	Load R0 Indexed R0=MEM({Rn+1,Rn})
F8	LDO	Rn, offset	4	NZ	Load R0 Indexed with offset R0=MEM({Rn+1,Rn}+offset)

Note that most opcodes insert the operand register number into the low 3 bits of the opcode.
 Rn = 3 bit register that goes in the REG field of the instruction.
 # = 3 bit encoded value of the bit being operated on.
 addr = 16 bit address
 offset = 8 bit relative offset, +127 to -128 for BR0 and BR1
 offset = 8 bit unsigned offset, 0 to 255 for STO and LDO
 value = 8 bit value

6.9 Addressing Modes

There are six addressing modes in the V8. The byte following a 2 byte opcode is typically called IAL. The 2 bytes following a 3 byte opcode are typically called BAL and BAH. Standard assembly notation is used here. A number in parenthesis indicates that the contents of the location pointed to by the number are to be used. Thus, (0x12) indicates the contents of address 0x12. A comma in the address is used to indicate the high and low byte of an address. Thus, (0x01,0xaa) indicates the contents of address 0x1aa.

- REG Register to Register - The Registers in the register file supply the operands.
- REL Relative - Branches are +127 or -128 bytes from the current value in the Program Counter.
- IMM Immediate - The byte following the opcode is the data.
- ABS Absolute - As the name suggests, two bytes follow the opcode to give the absolute address of the data. The byte following the opcode forms the low 8 bits of the address and the second byte is used as the upper 8 bits.
- INX Indexed - A pair of registers in the register file are used as a 16 bit index into memory. The register pair specified must always be even (R4 and R6 are commonly used). The even register contains the low 8 bits of the address and the Rn+1 register contains the high 8 bits.
- INO Indexed with offset - This mode is very similar to Indexed addressing except that an additional 8 bit offset is provided as an offset from the index. This addressing mode is often used by C compilers to allocate memory on the software stack. Typically R6,R7 are used as the software stack pointer.

6.10 Memory Map

The memory map of the V8 is completely programmable. However, the map below is the typical layout.

Address	Description
0000-01FF	BIST/BOOT ROM. Program execution begins at address 0x0000 after reset.
0200-02FF	Hardware Registers
0300-03FF	Stack Page
0400-0401	Non Maskable Interrupt vector (0400=PC[7:0], 0401=PC[15:8])
0402-0403	Maskable Interrupt INT(1) vector
0404-0405	Maskable Interrupt INT(2) vector
0406-0407	Maskable Interrupt INT(3) vector
0408-0409	Maskable Interrupt INT(4) vector
040A-040B	Maskable Interrupt INT(5) vector
040C-040D	Maskable Interrupt INT(6) vector
040E-040F	Maskable Interrupt INT(7) vector
0410-7FFF	General Purpose RAM Typically interrupt service routines and often-used functions. Typically the low 32K bytes are implemented in a single on-chip RAM. This RAM is usually initialized from an off-chip serial EEPROM at reset to provide the interrupt vectors and service routines.
8000-FFFF	General purpose RAM/Paged off-chip RAM This space is often divided up into various sizes depending on the application.

6.11 Instruction Set Details

Below is a detailed list of each opcode and the operation that is being performed during each clock. Cycle 0 is the opcode fetch and is overlapped with the completion of the previous opcode.

6.11.1 ADD - ADD

ADD = 50

$R0 \leq R0 + Rn$

Add R0 and Rn without carry

cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1					a+b	NZC			R0	Rn	add R0+ Rn and store the result in R0

Description:

Add Rn to R0 but ignore the Carry bit.

$N = R0[7]$

$Z = 1$ if $R0 = 0$

$C = 1$ if $R0 + Rn > 255$

Example:

lda r4, low ; This example shows how to add an 8 bit value to a 16 bit pointer

lda r5, high

lda r0, increment

ADD R4 ; Add R0 + R4

t0x R4

xor r0 ; clear R0 to zero

adc r5 ; add the carry bit to R5

t0x r5 ; store the result back in R5

6.11.2 ADC - Add with Carry

ADC = 08

$R0 \leq R0 + Rn + C$

Add Rn to R0 with Carry

cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1					a+b+C	NZC			R0	Rn	add a+b+carry and write to R0

Description:

Adds Rn plus the C bit to R0.

$N = R0[7]$

$Z = 1$ if $R0 = 0$

$C = 1$ if $Rn + R0 + C > 255$

Example:

ldi r4, 0x99 ; load R4 with 0x99

ldi r0, 0x55 ; load R0 with 0x55

stp 1 ; Set the carry bit

adc r4 ; $R0 = 0x99 + 0x55 + 1 = 0xEF$

6.11.3 AND - Logical AND

AND = 20		R0<=R0&Rn				Logical AND R0 and Rn					
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1					a&b	NZ			0	Rn	Logical AND R0 and Rn and write to R0

Description:

Logically AND Rn with R0.
 $N=R0[7]$
 $Z=1$ if $R0=0$

Example:

ldi r0, 0xaa
 ldi r1, 0xf0
 and r1 ; $R0=0xaa \& 0xf0 = 0xa0$

6.11.4 BR0 - Branch if 0

BR0 = 90		PC<=PC+Offset IF bit Rn of the PSR is 0				Branch if PSR bit is 0					
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		PC				+				opcode fetch
1	offset		PC		PSR&DCD		+				AND the PSR with the decode of Rn, load the offset into DL
2							*1				Load the PC if the Z flag from the ALU is 1
3											Use the new PC to fetch the next opode

*1 PC is loaded with the new value if the branch condition is true.

Description:

Branch if the desired bit of the PSR is zero.

Example:

CLP 0 ; Clear the Zero bit of the PSR
 BR0 0, always ; Branch is taken

Typically the following branch aliases are provided in the V8ASM.SYN file for easier reading:

BREQ <label>	= BR1 0, <label>	; Branch if Equal, Z bit set
BRNE <label>	= BR0 0, <label>	; Branch if Not Equal, Z bit zero
BRCC <label>	= BR0 1, <label>	; Branch if Carry bit Clear (zero)
BRLT <label>	= BR0 1, <label>	; Branch if CMP result is $R0 < Rn$
BRCS <label>	= BR1 1, <label>	; Branch if Carry bit Set
BRGE <label>	= BR1 1, <label>	; Branch if CMP result is $R0 \geq Rn$
BRPL <label>	= BR0 2, <label>	; Branch if Positive, N bit clear
BRMI <label>	= BR1 2, <label>	; Branch is Minus, N bit set

6.11.5 BR1 - Branch if 1

BR1 = 98

PC<=PC+Offset IF bit Rn of the PSR is 1

Branch if PSR bit is 1

cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		PC				+				opcode fetch
1	offset		PC		PSR&DCD		+				AND the PSR with the decode of Rn, load the offset into DL
2							*1				Load the PC if the Z flag from the ALU is 0
3											Use the new PC to fetch the next opode

*1 PC is loaded with the new value if the branch condition is true.

Description:

Branch if the desired bit of the PSR is a one.

Example:

```
STP    4          ; set PSR[4]
BR1    4, always  ; Branch to "always" is taken in this case as PSR[4]=1
```

See the branch aliases described in the BR0 opcode.

6.11.6 BTT - Bit Test

BTT = 60

Z<=~(R0&DCD)

Bit Test R0

cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1					a&DCD	NZ			R0		Set the Z bit of the PSR with the inverse of the bit specified in the Rn field in register R0

Description:

Test the desired bit and place it's inverse in the Z flag. This is done by anding R0 with a constant that is all zeroes except for a one in the bit position we wish to test.

N=R0[7]&DCD

Z=1 if R0[n]=0

Example:

```
BTT    4          ; Bit test bit 4 of R0
breq    one       ; Branch if R0[4]=0
```

6.11.7 CLP - Clear PSR bit

CLP = 68		PSR<=PSR&~DCD				Clear bit in PSR					
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1					PSR & ~DCD	L			R0		Clear the bit specified in the Rn field in PSR

Description:

Clear the desired bit of the PSR.

Example:

CLP 1 ; clear the Carry bit of the PSR (PSR[1])

6.11.8 CMP - Compare

CMP = 78		R0-Rn				Compare R0 with Rn (R0-Rn)					
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1					a-b	NZC			0	Rn	Subtract Rn from R0 and update flags

Description:

Compare R0 with Rn by subtracting Rn from R0, updating the flags but not storing the result into R0. The ALU performs a compare by performing a ones-complement (invert) on Rn and then adding R0 to Rn with the carry bit forced to one.

N=R0[7]

Z=1 if R0=0

C=1 if R0>=Rn

Example:

```

CMP    R1          ; compute R0-R1
BRCS   gt_or_eq    ; Branch if R0>=R1
BRCC   less        ; Branch if R0<R1
BREQ   equal       ; Branch if R0=R1
BRNE   not_eq      ; Branch if R0!=R1

```

6.11.9 DEC - Decrement

DEC = 40		Rn<=Rn-1				Decrement Rn by 1						
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description	
0	opcode		pc				+				opcode fetch	
1					a-1	NZC			Rn		subtract 1 from Rn and write to Rn	

Description:

Decrement Rn by 1. Note that in all subtraction modes, the C bit becomes a "NOT BORROW" bit. The ALU performs the decrement by computing a ones-complement (invert) on the constant 0, clears the carry bit and then performs an add operation. Thus, we add a 0xFF to the value in Rn which has the same effect as decrementing Rn. Note that the C bit is always set except when Rn=0 (this is the only time that $Rn+0xFF \leq 255$). We call the C bit a NOT BORROW for subtraction because it is zero when there is a borrow, and one when there is not a borrow.

$N = Rn[7]$

$Z = 1$ if $Rn = 0$

$C = 1$ if $Rn + 1 > 255$

Example:

```
ldi r1, 0x00
```

```
dec r1 ; R1=0x00-1=0xFF
```

6.11.10 INC - Increment

INC = 00		Rn<=Rn + 1				Increment Rn by 1						
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description	
0	opcode		pc				+				opcode fetch	
1					a+1	NZC			Rn		add 1 to Rn and write to Rn	

Description:

Increments register Rn by 1.

$N = Rn[7]$

$Z = 1$ if $Rn = 0$

$C = 1$ if $Rn + 1 > 255$

Example:

```
ldi r5, 0xff ; Load R5 with 0xff
```

```
inc r5 ; R5=0xff+1=0x00
```

6.11.11 INT - Interrupt

INT = A8 PC pushed onto stack, PSR pushed onto stack, I bit set.											Interrupt
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch Note that the PC is NOT incremented when an actual interrupt is executed.
1								-			Decrement stack to point to the next available location
2		PSR	SP	1				-			Write the PSR to the stack. Decrements stack pointer.
3		PCH	SP	1				-			Write PCH to the stack. Decrement the stack pointer.
4		PCL	SP	1			L				Write PCL to the stack, Load interrupt vector into PC
5	PCL						+				Fetch interrupt service routine address low 8 bits to DL
6	PCH						L				Fetch interrupt service routine address high and load PC
7											Use the new PC

Description:

Execute an interrupt service routine. When an interrupt is pending, this opcode is forced into the opcode register if the I bit is zero. The PSR and PC are pushed onto the stack, the I bit is set and a new PC is loaded from the interrupt vector table. Software can directly execute interrupt service routines simply by executing this opcode. When this opcode is executed in normal program flow, the I bit is ignored and the interrupt service routine is executed. The Program Counter is incremented when the INT opcode is executed in software. The Program Counter is **not** incremented when a hardware interrupt causes the INT opcode to be forced into the opcode register. This insures that the correct opcode is executed upon return from the interrupt service routine. INT 0 is normally reserved as a "break" opcode used for software breakpoints, single stepping or other debugging operations.

Example:

INT 0 ; Interrupt 0 is the non-maskable interrupt and is normally reserved for
 ; debugging purposes.

6.11.12 JMP - Jump Absolute

JMP = BC PCL<=(PC), PCH<=(PC+1). Jump to new address

cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1	PCL		pc				+				Fetch PCL to holding reg
2	PCH										Fetch PCH, load PC
3											

Description:

Jump to the specified address.

Example:

```

    brne    continue    ; branch around the long jump
    JMP     error_routine ; jump to an error routine which is often out of reach of +127/-128
continue:    ; range of a relative branch

```

6.11.13 JSR - Jump to Subroutine

JSR = BF PC<=new PC, PC+1 is pushed onto the stack. Jump to Subroutine

cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1	PCL		pc				+	-			PCL is saved in the PCL holding register. Decrement stack pointer to the next available location.
2		PCH	SP	1				-			The high 8 bits of the PC are pushed onto the stack. The stack pointer is decremented.
3		PCL	SP	1							The low 8 bits of the PC are pushed onto the stack.
4	PCH		pc				L				The new PC value is loaded. The low 8 bits comes from the PCL holding register with data fetched in cycle 1.
5											Use the new PC

Description:

Push the current PC onto the stack and Jump to the Subroutine at the address given in the next two bytes. Note that the return address that is pushed onto the stack does NOT point to the next opcode to be fetched. Instead it points to the return opcode -1. The RTN opcode will add the extra 1 count when the subroutine completes.

Example:

```

    JSR     my_subroutine

```

6.11.14 LDA - Load Absolute

LDA=E8		Rn<=(ADR)		Load Rn from absolute address							
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		PC				+				opcode fetch
1	ADRL		PC				+				DL<-ADRL
2	ADRH		PC								Fetch upper 8 bit of addr
3	data		D				0				Read RAM and store in DL
4					DL+0	NZ			Rn		Write DL to Rn

Description:

Load Rn with the value at the specified 16 bit address.

N=Rn[7]

Z=1 if Rn=0

Example:

LDA R4, 0x1234 ; load R4 with the contents of address 0x1234

6.11.15 LDI - Load Immediate

LDI=E0		Rn<=data		Load Rn with Immediate data							
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		PC				+				opcode fetch
1	data		PC				+				Fetch immediate data
2					DL+0	NZ			REG		store in Rn

Description:

Load Rn with the following byte of immediate data.

N=Rn[7]

Z=1 if Rn=0

Example:

LDI R4, 0xAA ; load R4 with 0xAA

6.11.16 LDO - Load Indexed with Offset

LDO=F8		R0<=([Rn+1,Rn]+offset)				Load R0 Indexed with offset					
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		PC				+				opcode fetch
1	Offset		PC				+				Fetch Offset and store in DL
2									REG	REG+1	Add Offset to D
3	data		D								Read RAM and store in DL
4					DL+0	NZ			0		Write DL to Rn

Description:

Load R0 with the contents of the address in the Rn/Rn+1 plus the offset. Note that the offset is an unsigned number in the range from 0 to 255.

N=R0[7]

Z=1 if R0=0

Example:

ldi r4, 0x34

ldi r5, 0x12

LDO R4, 0x80 ; load R0 with the contents of 0x12B4

6.11.17 LDX - Load Indexed

LDX=F0		R0<=(Rn+1,Rn)				Load R0 Indexed					
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1									REG	REG+1	Move REG pair to D
2	data		D								Read RAM and store in DL
3					DL+0	NZ			0		Write DL to Rn

Description:

Load R0 with the value at the address in Rn/Rn+1 register pair.

N=R0[7]

Z=1 if R0=0

Example:

ldi R4, 0x34

ldi R5, 0x12

LDX R4 ; load R0 with the value at 0x1234

6.11.18 OR - Logical OR

OR = 18		R0<=R0 Rn		Logical OR R0 with Rn							
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1					a b	NZ			R0	Rn	Logical OR R0 and Rn and write to R0

Description:

Logically OR Rn with R0.
N=R0[7]
Z=1 if R0=0

Example:

```
LDI    R1,0xa5      ; load R1 with 0xa5
LDI    R0 0x0f      ; Load R0 with 0x0f
OR     R1            ; R0 = 0x0f | 0xa5 = 0xa5; N=1, Z=0
```

6.11.19 POP - POP from Stack

POP = 88		Rn<=(SP). SP<=SP+1.		POP top of stack to Rn							
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1											Decode Opcode
2	Rn		SP					+			Load Rn from top of the stack. Increment stack
3					DL+0						Write data to Rn

Description:

Pop the top of the stack into Rn.

Example:

```
POP    R1          ; Pop the value on the top of the stack and store into Register R1
```

6.11.20 PSH - Push to Stack

PSH = 80		SP<=SP-1. (SP)<=Rn.		Push Rn onto top of stack							
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1								-			Decrement stack pointer
2		Rn	SP	1							Write Rn to the top of the stack.
3											

Description:

Push Rn onto the stack.

Example:

```
psh    r4          ; Push Register R4 onto the top of the stack.
```

6.11.21 ROL - Rotate Left

ROL = 30		Rn<=Rn<<1, C<=Rn[7]				Rotate Rn Left thru C					
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1					a<<1	NZC			Rn		Rotate Rn left 1 bit, C=Rn(7),Rn(0)=C

Description:

Rotate Rn left through the C bit.
N=Rn[7]
C=Rn[7] before the shift occurs

Example:

```
LDI R0, 0xff    ; R0=0xff
CLP 1          ; clear the carry bit
ROL R0         ; R0=0xfe, C=1, N=1, Z=0 after execution
```

6.11.22 ROR - Rotate Right

ROR = 38		Rn<=Rn>>1, C<=Rn[0]				Rotate Rn Right thru C					
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1					a>>1	NZC			Rn		Rotate Rn right 1 bit, C=Rn(0),Rn(7)=C

Description:

Rotate Rn right through the C bit.
N=Rn[7]
C=Rn[0] before the shift occurs

Example:

```
LDI R1, 0      ; R0=0
STP 1          ; set the carry bit
ROR R1         ; R0=0x80, C=0, N=1, Z=0 after execution
```

6.11.23 RTI - Return from Interrupt

RTI = BA		PC<=PC from stack. PSR<=PSR from stack.						Return from Interrupt			
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1											Decode Opcode
2	PCL		SP					+			Load PCL from stack to PCL holding register. Increment stack
3	PCH		SP					+			Load the entire PC. The stack pointer is incremented.
4	PSR		SP			L		+			Load the PSR from the stack and save in DL. The stack pointer is incremented.
5					DL+0						load PSR from DL

Description:

Return from Interrupt. This is basically the same as the RTS opcode except that the PSR is also popped off of the stack.

Example:

RTI ; normally the RTI opcode is simply the last instruction in an interrupt service routine

6.11.24 RTS - Return from Subroutine

RTS = B9		PC<=PC from SP + 1. SP<=SP+2						Return from Subroutine				
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description	
0	opcode		pc				+				opcode fetch	
1											Decode Opcode	
2	PCL		SP					+			Load PCL from stack to PCL holding register. Increment stack	
3	PCH		SP				L	+			Load the entire PC. The stack pointer is incremented.	
4							+				Since the JSR pushed the PC-1, we have to increment it here.	
5											Use the new PC	

Description:

Return from subroutine by popping the PC value from the stack. Note that the return address popped from the stack is incremented before it is loaded into the PC.

Example:

RTS ; in most cases, the RTS opcode is simply the last opcode in a subroutine.

```

; This example shows how to use the RTS opcode to perform a jump table operation
; It is assumed that the index to the desired table location is in the R4/R5 register pair.
ldx    r4    ; load the high byte of the jump table
psh    r0    ; push onto the stack
ldo    r4,1  ; load the low byte
dec r0      ; decrement due to the increment in the RTS opcode, we assume that the table
              ; does not cross a page boundary
psh    r0    ; push onto the stack
RTS      ; Begin execution at the address computed in the jump table

```

6.11.25 SBC - Subtract with Carry

SBC = 48		R0<=R0-Rn-C				Subtract Rn from R0 with carry						
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description	
0	opcode		pc				+				opcode fetch	
1					a-b-1+C	NZC			R0	Rn	Subtract Rn from R0 with carry	

Description:

Subtract Rn from R0 with carry. Note that during subtraction, the Carry bit becomes a NOT-BORROW. Thus, setting the C bit causes no borrow, and clearing the C bit causes a borrow.

N=R0[7]

Z=1 if R0=0

C=1 if R0>=Rn+C

The hardware operation performed when doing a subtract is:

$\{C, alu_out, X\} = \{R0, 1\} + \{\text{NOT}(Rn), C\}$

Where {} is a concatenation operator. Note that the two operands on the right side of the equation are 9 bits wide. The Rn operand is one-complemented (inverted). The Carry is concatenated to the LSB of the inverted Rn and a one is concatenated to the LSB of R0. Taking the ones-complement in effect subtracts Rn-1 from R0, the Carry is then added back into the equation. The left side of the equation is 10 bits where the Carry out is the MSB and the LSB is discarded as it was used to simply get the carry bit into the equation. While this may seem a bit complicated, it is the lowest gate count method of doing a subtraction. No subtractor is needed, just an adder and 8 inverters, both of which are used in other operations anyway.

Example:

ldi r1, 0x12

ldi r0, 0x13

stp 1 ; setting C causes no borrow

sbc r1 ; R0=0x13 - 0x12 - 0 = 0x01, N=0,Z=0,C=1

6.11.26 STA - Store Absolute

STA=C8		(ADR)<=Rn				Store Rn to the specified absolute address						
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description	
0	opcode		PC				+				opcode fetch	
1	ADRL		PC				+				Fetch low address	
2	ADRH		PC								Fetch high address	
3		Rn	D	1			+			REG	Write Rn to RAM	
4												

Description:

Store Rn to the address specified.

Example:

ldi r1, 0x01

STA r1, FLAG ; store 0x01 to the address of the FLAG variable

6.11.27 STO - Store Indexed with Offset

STO=D8		([Rn+1,Rn]+offset)<=R0				Store R0 Indexed with offset					
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1	Offset		pc				+				load the offset
2									REG	REG+1	Add the offset to the index register pair
3		R0	D	1						0	Write R0 to RAM
4											

Description:

Store R0 to the address in the register pair Rn/Rn+1 plus an offset. The offset is an 8-bit unsigned number in the range from 0 to 255

Example:

```
ldi    R4, BASE_ADDR      ; load the low half of the pointer
ldi    R5, BASE_ADDR>>8   ; load the high half of the pointer
STO    R4, 0x03            ; Store R0 to BASE_ADDR+0x03
```

6.11.28 STP - Set PSR bit

STP = 58		PSR<=PSR DCD				Set bit in PSR					
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1					PSR DCD	L			R0		Set the bit specified in the Rn field in the PSR

Description:

Set the desired bit of the PSR.

Example:

```
STP    7      ; will set PSR[7] to a one
```

6.11.29 STX - Store Indexed

STX=D0		(Rn+1,Rn)<=R0				Store R0 Indexed						
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description	
0	opcode		PC				+				opcode fetch	
1									REG	REG+1	Move both registers to the address latch	
2		R0	D	1						0	Write R0 to RAM	
3												

Description:

Store R0 to the address in the register pair Rn/Rn+1.

Example:

```
ldi    R4, BASE_ADDR      ; load the low half of the pointer
ldi    R5, BASE_ADDR>>8   ; load the high half of the pointer
STX    R4                  ; Store R0 to BASE_ADDR
```

6.11.30 T0X - Transfer R0 to Rn

T0X = 70		Rn<=R0				Transfer R0 to Rn						
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description	
0	opcode		pc				+				opcode fetch	
1					0 b	NZ			Rn	R0	Move R0 to Rn	

Description:

Transfer R0 to Rn.
N=Rn[7]
Z=1 if Rn=0

Example:

```
T0X    R7                  ; Copy R0 to R7
```

6.11.31 TX0 - Transfer Rn to R0

TX0 = 10		R0<=Rn				Transfer Rn to R0						
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description	
0	opcode		pc				+				opcode fetch	
1					a 0	NZ			R0	Rn	Move Rn into R0	

Description:

Transfer Rn to R0.
N=R0[7]
Z=1 if R0=0

Example:

```
TX0    r1                  ; Copy R1 to R0
```

6.11.32 UPP - Increment Register Pair

UPP = C0 (Rn+1,Rn)<=(Rn+1,Rn)+1.

Increment Register Pair

cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1					a+1	C			Rn		Increment Rn (must be even)
2					a+C	C			R(n+1)		Increment R(n+1) if Carry

Description:

Increment the register pair Rn/Rn+1. The specified register must be even.

C=1 if the register pair is incremented from 0xFFFF to 0x0000. Cleared to 0 otherwise.

Example:

```

ldi    r4, 0x34
ldi    r5, 0x12
ldi    r6, 0xcd
ldi    r7, 0xab
lda    r1, NUM_BYTES_TO_MOVE
loop:  ldx    r4      ; This example moves a block of data starting at 0x1234 to 0xabcd
      stx    r6      ;
      UPP    r4      ; increment the R4/R5 source register pair
      UPP    r6      ; increment the R6/R7 destination register pair
      dec    r1      ; decrement the loop counter
      brne   loop    ; branch back until all the bytes have been copied

```

6.11.33 XOR - Logical Exclusive-OR

XOR = 28 R0<=R0^Rn

Logical Exclusive-OR R0 with Rn

cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1					a^b	NZ			R0	Rn	Logical XOR R0 and Rn and write to R0

Description:

Logically Exclusive OR Rn with R0.

N=R0[7]

Z=1 if R0=0

Example:

```

XOR    R0      ; Exclusive ORing R0 with itself will clear R0 to 0. N=0, Z=1
          ; Note that XORing a register with itself will NOT clear the register to zero
          ; for gate level simulations. The register will remain X. Always do a LDI r0,0
          ; early on in a simulation. The XOR will always work in the real world.

```

6.11.34 XSP - Exchange Stack Pointer

RSP = B8		SP<={R1,R0}, {R1,R0}<=SP						Reset the Stack Pointer			
cyc	d in	dout	addr	wr	alu	psr	pc	stack	rega	regb	description
0	opcode		pc				+				opcode fetch
1									R0		Setup
2					DL+0			R0	R0		Exchange R0
3									R1		Dead
4					DL+0			R1	R1		Exchange R1
5											Dead

Description:

The value of the Stack Pointer (SP) is exchanged with the register pair R1/R0. R1 will be exchanged with SP[15:8] and R0 is exchanged with SP[7:0]. Note that in most implementations of the V8, the upper 8 bits of the SP are fixed at 0x03. However, the size of the stack pointer is programmable and may be implemented either as a full 16 bit register to as little as a 4 bit register. This opcode has been added to specifically support Real-Time Operating Systems (RTOS) and it is strongly recommended that a full 16 bit SP be implemented in systems that expect to use an RTOS.

Example:

```
LDI r0, 0x02
LDI r1, 0x01
XSP          ; SP=0x0102 and R1=SP[15:8], R0=SP[7:0]
XSP          ; Two XSPs in a row are effectively a NOP ONLY if the SP is a full 16 bits.
```

7 Signal Description

Pin Name	Direction	Description
ADDR[15:0]	OUT	Address Bus
DATAIN[7:0]	IN	Data Input Bus
DATAOUT[7:0]	OUT	Data Output Bus
READ	OUT	Read Cycle High when data is being loaded from the DATAIN bus.
WRITE, WRITE_D	OUT	Write Cycle High when data is valid on the DATAOUT bus.
CLK	IN	Clock All Flip-Flops clock on the rising edge of the clock.
RESET	IN	Reset Must be asserted for at least 3 rising edges of CLK.
INT[7:0]	IN	Interrupt Request INT[0] is Non-maskable and is generally reserved for JTAG debugging purposes.
READY	IN	Ready Freezes the current state when low, operation completes when high. Note that the processor stops even if there is no memory operation currently in progress. Must be high when RESET is asserted.
SP_OFLO	OUT	Stack Pointer Over Flow
OP_FETCH	OUT	Opcode Fetch Cycle. Typically used for single step debugging.
CLR_P[7:4]	IN	Clear PSR bit
SET_P[7:4]	IN	SET PSR bit
PSR	OUT	Processor Status Register

7.1 Address Bus (ADDR[15:0])

The 16 bit address bus is used to address data in memory. The bus will have invalid addresses presented on it during internal operations of some opcodes. Neither READ nor WRITE will be active during these invalid cycles. The address comes directly from a register for rapid decoding.

7.2 Data Bus (DATAIN[7:0], DATAOUT[7:0])

The 8 bit data bus is separated into two separate busses. The DATAIN bus is typically from a multiplexor of on-chip RAM and peripherals or from the input buffer of a bidirectional tristate I/O cell for off-chip devices.

The DATAOUT bus is typically connected to the inputs of on-chip RAM and peripherals or to output drivers of a bidirectional tristate I/O cell for off-chip devices.

7.3 Write Enable (WRITE, WRITE_D)

WRITE indicates whether the microprocessor is writing to memory or an I/O device. When high (1), data is being output on the DATAOUT bus. WRITE is driven directly by the output of a flip-flop. Thus, it has minimal clock to output delay and is glitch free. WRITE should be used as a register enable by external devices to latch data on the DATAOUT bus. When using an external tristate bus, WRITE is used as a tristate enable. An external active low WE_N pulse is generated by gating the CLK with WRITE so that the write pulse is active during the second half of a clock period.

WRITE_D is the D input to the Flip-Flop that generates WRITE. Thus, WRITE_D is active one clock before WRITE. However, WRITE_D is a decoded signal and will glitch. Typically this signal is not used except by the V8_DEICE core. The V8_DEICE core uses the WRITE_D signal to combine its own WRITE signal without adding any additional gate delays to the critical WRITE signal. This insures that the WRITE signal comes directly from a DFF even when the V8_DEICE core is used.

7.4 Read Enable (READ)

READ indicates that the microprocessor is reading from memory or an I/O device. When high (1), data is expected on the DATAIN bus. READ is a decoded signal and thus may glitch.

7.5 RESET

RESET forces OP_FETCH=1, PC=0, PSR=0x08, SP=default value. Execution begins at 0x0000 and the I bit of the PSR is set high to prevent interrupts from being processed. READY must be high when RESET is asserted.

7.6 Interrupts (INT[7:0])

The INT[7:1] pins are maskable interrupts to the processor. A high (1) on INT[7:1] during an opcode fetch will trigger the interrupt sequence if the I bit of the PSR is low (0). The INT[7:1] signals are **not** synchronized to the rising edge of the clock and must be made synchronous before applying them to the V8. The interrupt sequence begins by forcing an INT opcode into the instruction register. The PSR and PC are pushed onto the stack. The I bit of the PSR is set. The INT interrupt vector is loaded into the PC and program execution begins at the new PC. The I bit being a 1 prevents other assertions of INT from being recognized until the current interrupt is serviced. Note that INT is level sensitive and requires that both the I bit be low and that INT be active while an opcode is fetched (OP_FETCH=1) in order for the interrupt to be recognized.

INT[0] is a non-maskable interrupt. It is level sensitive just like the maskable interrupts. Because it is level sensitive, the logic driving this signal **MUST** force INT[0] to go low after the clock edge when INT[0]=1 and OP_FETCH=1. INT[0] must remain low until the non-maskable interrupt has been serviced. Since INT[0] is non-maskable, once the interrupt has been recognized (OP_FETCH and INT[0] being high) the interrupt processing will begin. If the INT[0] line continues to be asserted after OP_FETCH has been high, then the interrupt service routine will be entered multiple times resulting in unpredictable operation.

The interrupt lines are priority encoded with INT[0] have the highest priority and INT[7] having the lowest priority. Thus, when multiple interrupts are pending, the interrupt with the highest priority (the lowest index) will be serviced. For example, if INT[3], INT[5] and INT[6] signals are all high, then INT[3] will be serviced first, followed by INT[5] and finally INT[6]. Note that this scheme is quite simple and low priority interrupts can easily be "starved" if there are too many higher priority interrupts. The V8 core can be easily modified or external logic added to provide round-robin or other fairness based priority algorithms for those applications that need them.

The eight separate interrupt lines keep the interrupt latency low in the standard form of the V8. Each interrupt has its own interrupt vector so the source of the interrupt does not have to be polled or computed. You simply jump directly into the proper interrupt service routine.

The V8 registers are not automatically placed on the stack when an interrupt is detected. It is up to the programmer to save the registers either in memory or on the stack. The saving and restoring of the

registers does increase the interrupt latency. However, the interrupt latency time can be dramatically reduced by providing a second bank of registers that are used only for interrupt processing. See section 6.4 on page 7 for more details on providing separate register banks for interrupt processing.

7.7 Ready (READY)

READY is used to hold the microprocessor in the current state for multiple clock cycles. This is most often used to insert wait states when accessing slow peripherals. READY is normally high indicating that the system is ready and the processor can continue. When READY is low (0), the processor stops execution at the current state. The processor stops at the current state even if both READ and WRITE are inactive due to an internal processing cycle. When READY is asserted, the internal signals of the V8 will be static except for the clock. Thus, READY can be used to reduce power consumption while waiting for external stimulus. READY must be high when RESET is asserted.

Since READY passes through the Opcode Decode block, it typically has a significant amount of delay. It is recommended that this signal come directly from a flip-flop or that it be optimized when synthesizing.

7.8 Set/Clear PSR Bits (SET_P, CLR_P)

The SET_P and CLR_P signals can be connected to external devices or IO lines which must be polled rapidly. These signals provide direct access to the upper 4 bits of the PSR. A high on the SET_P signal will cause the corresponding bit in the PSR to be set. A high on the CLR_P signal will cause the corresponding PSR bit to be cleared to zero. Separate controls are provided to allow maximum flexibility. By connecting a status signal to the SET_P and its inverse to CLR_P the signal value can be directly tested at anytime. If the status signal is a short pulse, it can be connected to either SET_P or CLR_P and the other side connected to ground. The pulse will then set or clear the PSR bit which software can then detect and clear or set when it has processed the pulse.

7.9 Stack Pointer Overflow (SP_OFLO)

SP_OFLO goes high (1) whenever the stack pointer rolls over. Typically this indicates that the stack pointer has overflowed and data on the stack has been lost. Since data has been lost, the only usual solution is to effectively reset the processor and reload the SP. This signal is normally synchronously sets a flip-flop which has the Q output connected to the NMI interrupt (INT(0)). OP_FETCH and READY synchronously clears the flip-flop. This flip-flop will then cause a Non-Maskable interrupt when the stack overflows. This condition should be clearly reported to the software developer and the processor should be reset. Under normal operation, the stack should never overflow.

7.10 Opcode Fetch (OP_FETCH)

OP_FETCH goes high (1) whenever an opcode is fetched from memory. It can be combined with READY to cause single cycle execution for debugging purposes.

7.11 Processor Status Register (PSR)

The Processor Status Register is driven out of the V8 core as a debugging aid. These signals can either be bonded directly to IO pins if available or connected to other special functions.

8 Timing

All timing parameters are relative to the rising edge of CLK. All input signals are sampled at the rising edge of CLK and all output signals transition after some delay relative to the rising edge of CLK. Input Hold times are the amount of time after the rising edge of the clock that a signal must remain stable. Output Hold times are the minimum delay that the signal will remain stable after the rising edge of the clock. Note that most of the outputs are decoded and thus will glitch during the period from the hold time to the delay time.

Since this is a synthesizable model and not an actual IC, actual timing parameters cannot be provided. Timing parameters will vary significantly depending on the synthesizer used to generate the logic, the constraints provided to the synthesizer and the target technology. **Figure 3** shows a read and a write cycle and provides timing parameters for your reference. The chart below provides some relative delay times when the core is synthesized to LSI Logic G10, 0.35 micron, 3.3V, CMOS technology. Use these numbers only as a guide. Your actual delay times may vary significantly. There is an extra column in the chart where you can include the delays from your implementation of the V8.

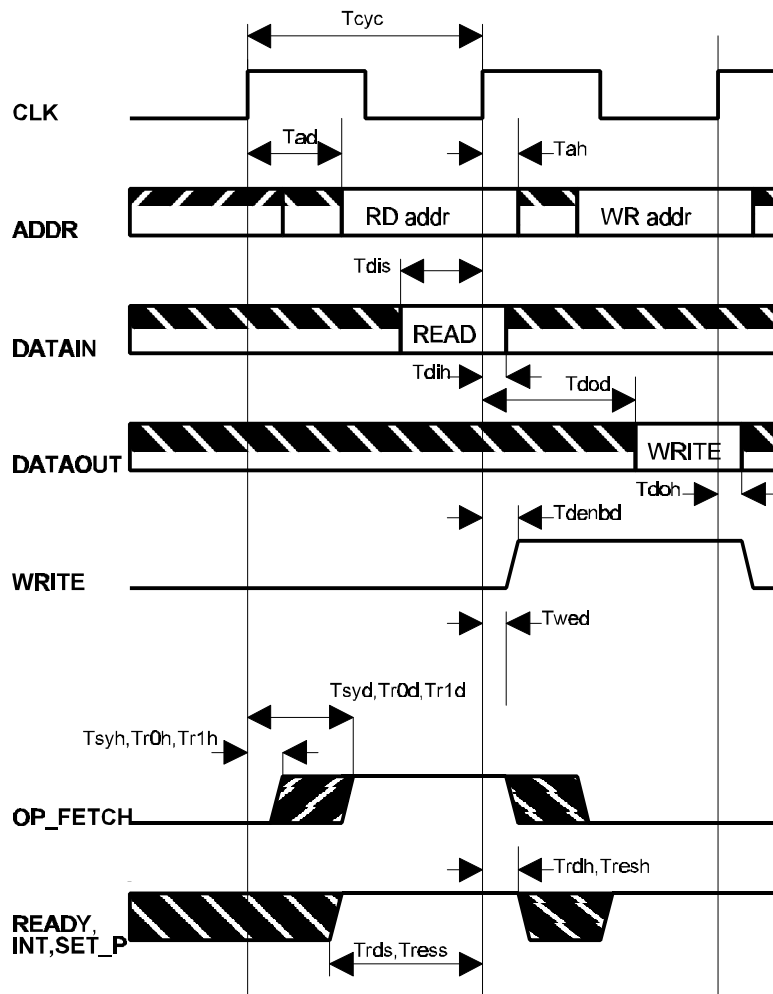


Figure 3

Parameter	Description	Value		Units
Tcyc	Clock cycle time (min)	12.56		nS
Tad	ADDR max delay	0.42		nS
Tah	ADDR min hold	0.34		nS
Tdis	DATAIN setup	0.77		nS
Tdih	DATAIN hold	0		nS
Tdod	DATAOUT max delay	6.38		nS
Tdoh	DATAOUT min hold	0.69		nS
Twed	WRITE delay	0.47		nS
Tred	READ delay	3.83		nS
Tofh	OP_FETCH delay	3.54		nS
Tints	INT setup	7.66		nS
Tinth	INT hold	-0.52		nS
Tscs	SET_P,CLR_P setup	0.35		nS
Tsch	SET_P,CLR_P hold	0		nS
Trds	READY setup	2.62		nS
Trdh	READY hold	0		nS
Tress	RESET setup	5.82		nS
Tresh	RESET hold	0		nS

9 V8 Standard Peripherals

The V8 comes with a number of standard peripherals. Details on signals and interfacing requirements can be found in the HDL source code files and in the ASIC_V8 module. The descriptions that follow are meant primarily to describe the operation of the peripheral from a software viewpoint. VAutomation has numerous other peripherals or variations of these peripherals which have been developed over the years. Contact VAutomation if you don't see the peripheral you need. We may already have it in our library or we can easily customize a peripheral to exactly meet your needs.

Unless otherwise specified, all registers are typically reset to zero when the RESET signal is active.

9.1 ASIC_V8

The ASIC_V8 module provides a complete example of how to assemble a complete system on a chip solution with the V8. The ASIC_V8 is mostly a structural model that links the various synthesizable cores in the following sections together. Some glue logic is provided as well as 512 bytes of BIST/BOOT ROM. See the HDL Source code file for more details on all of the logic provided. Typically another module that instantiates the IO cells is placed just above this module. In the IntelliCore Prototype System example provided, the XIL240HQ or A10k240 module instantiates the ASIC_V8, IO cells, FPGA clock buffers and the gate for the off-chip write enable. Use the ASIC_V8 module as a guide to develop your own system on a chip ASIC or FPGA.

All of the peripherals connect to the V8 address and data busses. Control signals are active high. All of the peripherals are fully synchronous to the V8 so the control signals, address and data lines need only be active for a single clock.

The ASIC_V8 develops the following address map:

- 0000-01ff = BIST/BOOT ROM. CPU begins execution at 0x0000 upon reset

- 0200-02ff = Hardware Registers

 - 0200-0200 = Page register

 - 0202-0202 = Control register (write only)

 - CTL_REG[7] = unused

 - CTL_REG[6] = unused

 - CTL_REG[5] = Unused

 - CTL_REG[4] = Unused

 - CTL_REG[3] = Interrupt Enable. 0=disable all interrupts,1=enable

 - CTL_REG[2:0] = PSR control select

 - 000 = PSR[7:4] under software control only

 - 001 = Enable I2C bus

 - xxx = Other combinations are not defined.

 - 0210-0213 = External Audio chip (AD1845)

 - 0220-0223 = 1284 parallel port interface

 - 0230-0233 = Power Save Unit

 - 0240-0242 = VUART serial communications controller

 - 0250-0253 = Reserved

 - 0260-0263 = VTIMER 16 bit counter timer

 - 0280-02ff = External peripherals

- 0300-03ff = STACK

- 0400-040f = Interrupt vectors

- 0410-7fff = RAM

- 8000-ffff = Off-chip RAM window selected by the PAGE register

A15 enables the extended addressing PAGE register. The upper 32K bytes of the V8s address space provides a "window" into the 8 Megabyte address range. The 23 bit space is further divided into SRAM and DRAM spaces: 000000-01ffff=SRAM, 080000-7fffff=DRAM. Note that accesses to DRAM take several cycles for each access.

9.2 VUART 1000 gates

The VUART module is a very simple UART with an integrated BAUD rate generator. The UART only supports 8 data bits, 1 start bit and 1 stop bit with no parity. This is the most common mode required for most embedded applications. Several other variations of UARTs are available from VAutomation.

A programmable BAUD rate generator is provided which generates a 4X oversampled bit clock and a 1X bit clock. The baud rate registers are write only. To compute the value to be programmed into the BAUD registers, use the following equation:

$$\text{BAUD} = (\text{CLK_FREQ}/\text{BAUD RATE}/4)-1$$

Example: if the desired baud rate is 115200 baud and the clock frequency of the V8 is 12Mhz:

$$\text{BAUD} = (12,000,000/115200/4)-1 = 25.04 = 0x19$$

Often the clock frequency of the V8 is not an even multiple of the desired BAUD rate. Typically you will want to round the value to the nearest whole integer. This will introduce some error in the BAUD rate, but at BAUD rates below 57.6K the error is negligible.

The UART is full duplex capable and both the transmit and receive channels effectively have a single byte FIFO built in. To transmit a byte, simply write to the DataTX register. This will cause the TXEMPTY bit in the STATUS register to go low. Once the byte has been transferred from the holding register into the shift register, TXEMPTY will go high indicating that another byte can be written to DataTX. Typically, once all bytes have been written to the UART, the TXENB bit is cleared to zero which masks the interrupt until more data is available for transmission.

To receive data, simply enable RX interrupts by setting RXENB to a one. RXFULL will go to a one when a character has been received and the interrupt will be activated if RXENB is a one. Reading DataRX will clear RXFULL. If another character is received before the previous one has been read (RXFULL is still a one), then the RXOFLO bit is set indicating that data has been lost. If there is a framing error, the RXFRAM bit is set. RXOFLO and RXFRAM must be cleared by software by writing a zero to these bits.

Register Definition:

ADDR	NAME	R/W	DESCRIPTION
0	DataRX	R	Receive data holding register
0	DataTX	W	Transmit data holding register
1	STATUS	R/W	Status Register: [7]=TXEMPTY (read only) 1=DataTX can be written with the next character, 0=transmission in progress. Cleared to zero when a write to DataTX is performed. [6]=TXENB 1=enable TXEMPTY to activate IRQ, 0=masked [5:4]=reserved, write zeroes. [3]=RXFULL (read only) 1=Data available in DataRX, 0=no data available. Cleared to zero when DataRX is read. [2]=RXENB 1=enable RXFULL or RXOFLO or RXFRAM to activate IRQ, 0=masked. [1]=RXOFLO 1=character received but RXFULL was still high. [0]=RX framing error 1=STOP bit not detected.
2	BAUDL	W	Baud Rate Register Low [7:0]
3	BAUDH	W	Baud Rate Register High [15:8]

9.3 VTIMER 1600 gates

The VTIMER module is a complete 16-bit counter/timer with Pulse-Width-Modulation capabilities. Typically the PRESCALE, MAXA and MAXB registers are programmed with the desired count rate, then the RUN bit in the CTL register is set to one and the counter begins counting up from zero. When the CTR register equals the value in MAXA, the INT bit is set and an interrupt is asserted if INTENB=1. If RET is a zero, then the counter will stop and the RUN bit is cleared. If RET is a one, then the counter is cleared to zero and begins counting again. If MXB is a one, then the counter will count up to the value programmed in MAXB. If MXB is a zero, the counter will ignore MAXB and use only MAXA.

The CTR registers can be read or written at anytime but care must be taken when reading as the counter may be running and the counter could increment between reads of CTRL and CTRH.

The counter has an external enable signal called EXT_ENB. This signal is a level sensitive signal which enables the counter to count when the signal is high. EXT_ENB **must** be synchronized to the CLK signal. To make EXT_ENB edge sensitive instead of level sensitive, you must clock EXT_ENB into two flip-flops and gate their outputs together to create a one CLK wide pulse. Note that EXT_ENB must be at least one-half the frequency of CLK.

All registers are cleared to zero when RESET is asserted.

Register Definition:

ADDR	NAME	R/W	DESCRIPTION
0	CTRL	R/W	Counter value [7:0]
1	CTRH	R/W	Counter value [15:8]
2	CTL	R/W	Control Register [7]=INT MAX count reached. Write a one to clear this bit, writing a zero has no effect. [6]=INTENB 1=enable interrupts, 0=disabled. [5]=PWM 1=using the MAXB count, 0=using MAXA count. This bit is the same state as the external PWM signal. This bit is read-only. [4]=RET 1=retrigger when MAX reached, 0=stop when MAX reached. [3]=MXB 1=use MAXB, 0=use only MAXA [2]=Reserved, set to zero. [1]=EXT 1=use the external EXT_ENB signal to enable the counter, 0=use the internal clock. [0]=RUN 1=enable counter, 0=stop. Automatically cleared to zero when MAX reached is RET=0.
3	PRESCL	W	Prescale register Prescale the selected count enable by the value in this register. A value of zero disables the prescaler. A value of 1 performs a divide by 2, 2 will divide by 3, etc, 0xff will divide by 256.
4	MAXAL	W	MAXimum Count Value A [7:0] When the counter counts up to the value programmed in this register, an interrupt will be generated.
5	MAXAH	W	MAXimum Count Value A [15:8]
6	MAXBL	W	MAX B [7:0]
7	MAXBH	W	MAX B [15:8]

9.4 V1284

The V1284 module is a IEEE 1284 compatible Parallel Printer port interface. The interface supports only the IEEE 1284 Compatible mode. Some software support is required to perform some of the signaling required during bus enumeration. Operation basically consists of programming the CTRS and CTRH registers with the appropriate width pulses for STROBE_N. Typically, if the CLK is 12Mhz, each of the 4 bit fields should be programmed with a 0xb which will result in a 1 us time for each phase. Then data is simply written to DOUT, the hardware will handle the STROBE_N generation automatically, and software should poll the BUSY bit before writing another byte to DOUT.

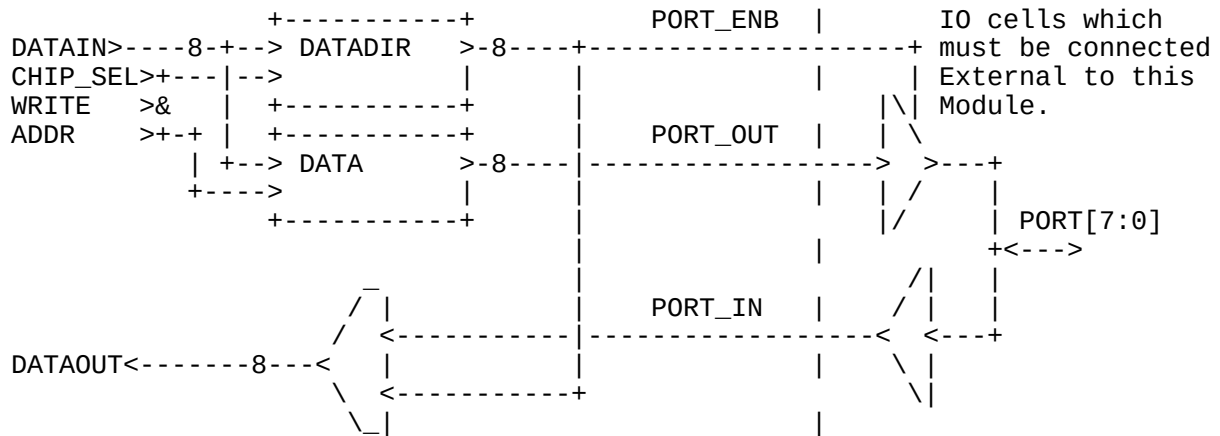
Register Definition:

ADDR	NAME	R/W	DESCRIPTION
0	DOUT	W	Parallel DATA out
0	DIN	R	Parallel DATA in
1	CTRS	R/W	CTRS[7:4] = Tsetup which is the amount of time that the data is stable before STROBE_N is driven active, 0=1 clock, 0xf=16 clocks. CTRS[3:0] = Tstrobe which is the length of the STROBE_N, 0=1 clock, 0xf=16 clocks.
2	CTRH	R/W	CTRH[7:4] = Thold which is the length of time that data is held stable after the deassertion of STROBE_N. 0=1 clk, 0xf=16clks CTRH[3] = reserved CTRH[2] = DRVDO This bit forces the tristate enable on the P1284 pins to the ON state. This allows the V1284 core to drive data onto these pins without activating the STROBE_N signal. CTRH[1] = STLOW This bit when 1 forces STROBE_N low. This provides a software controllable means of communicating with the peripheral. CTRH[0] = STDIS This bit when 1, disables the automatic STROBE_N generation logic. This bit is must be set before setting the STLOW bit.
3	CTL	R/W	Data register for the control signals
4	CTLZ	R/W	Data Direction register for the control signals. When the respective bit=1, the signal is driving, when zero, the signal is in tristate. The control bus is typically wired with the following 1284 Compatible signal names: [7]=init_n [6]=autofeed_n [5]=selectin_n [4]=ack_n [3]=busy [2]=perror [1]=select [0]nfault_n

9.5 VPPIO (400 gates)

The VPPIO module is a simple 8-bit Parallel Input/Output port. The VPPIO provides two 8-bit registers which in turn are connected to eight bidirectional IO pins which must be added external to this logic. The CPU interface is quite simple and is fully synchronous. The DataOUT register is a write-only register which holds the values to be driven onto the external IO pins. The DataDIR register defines whether each bit is an input or an output. The DataIN register is a read-only register which provides the live values on the external IO pins.

Block Diagram:



Register Definition:

ADDR	NAME	R/W	DESCRIPTION
0	DataOUT	W	Data value to be driven onto the IO pins
0	DataIN	R	Data value currently on the IO pins.
1	DataDIR	R/W	Data DIRection register Each bit controls the tristate enable of the corresponding bit of the IO pin. A one indicates that the IO pins is an output pin (DataOUT drives the IO pin). A zero defines the IO pin to be an input only pin and DataOUT has no effect.

9.6 PWR_SAVE Unit 400 Gates

The Power Save Unit provides four power saving modes of operation. All of the modes are entered by programming the registers. All of the low power modes are exited when any of the eight QUIET signals changes from it's default state to an active state. This logic requires support at the IO cell level for the all of the low power modes. See the PWR_SAVE.VHD file for an example on how to connect the outputs from this logic to gate the clock or stop the oscillator.

REST mode:

In REST mode, the REST output is active. REST is normally connected to the wait-state generator of the microprocessor (READY on the V8) and causes an infinite wait-state to be generated. This halts the microprocessor which reduces power consumption. All clocks continue to run so timers, UARTs and other peripherals can wake up the microprocessor.

SLEEP mode:

In SLEEP mode, the SLEEP signal is active high. Typically SLEEP is synchronized through three or more flip-flops and then gated with the clock signal just before it is distributed throughout the chip. UARTs and Timers will no longer function so only external logic will wake the processor. The oscillator continues to run as do the synchronizing Flip-Flops. This is a reliable low-power mode as the oscillator continues to run and is stable. Typically saves another 10-15%.

GLO_TRI mode:

GLO_TRI mode requires support from the IO cell level of the target chip. This mode causes all IO cells to go into tristate mode which in turn reduces current drain further. Care must be take to NOT tristate the oscillator signals and that all important external signals have pullup or pulldown resistors to insure external devices are also deactivated. Typically this mode is entered in addition to one of the other three modes.

STOP_OSC mode:

STOP_OSC mode also requires external support. If a crystal oscillator IO cell is used, this signal will gate the feedback (or output) cell which in turn stops the oscillator. This is the lowest power consumption mode and should be at the lowest CMOS leakage levels (IE: nanoamps). Care must be taken when restarting the oscillator to wait until it has stabilized before allowing the microprocessor to begin operating. It is recommended that the SLEEP bit always be set along with the STOP_OSC bit. The SLEEP bit will gate the clock to the rest of the ASIC for several clocks while the oscillator is stabilizing.

Register Definitions:

ADDR	NAME	R/W	DESCRIPTION
0	CTL	R/W	[0]=rest halt the microprocessor (clocks run) [1]=sleep stop the clock (oscillator runs 3 DFFS) [2]=GlobalTri Tristate all IO cells [3]=stop_osc stop the oscillator [4:7]= reserved, always zero
1	STATUS	R	Current state of the QUIET signals
2	POLRTY	R/W	Polarity of the inactive state of the QUIET signals
3	ENABLE	R/W	Each bit corresponds to the respective QUIET signal bit. A 1 enables the QUIET signal to exit the power save mode, a 0 masks the QUIET signal.

Typically, when software decides it's time to go to sleep, it will read the status register and decide if the QUIET signals are in their desired state. Write the value read from the STATUS register into the POLRTY register. This sets the current value of the QUIET signals to be the "quiet" state. Write the ENABLE register with the bits that you want to wake-up on being a one, all other bits should be zero. Finally, write the CONTROL register with a one in the sleep modes selected. For maximum power reduction, you'll want to set the all four bits. Typically REST mode is used in your applications "idle loop" when it is waiting for something to process. SLEEP mode is typically used when fine control over the oscillator is not available or you need to wake up quickly and cannot wait for the oscillator to stabilize. Generally you should set all of the power save modes up to the highest level you need. Specifically setting the SLEEP bit and the STOP_OSC at the same time is highly recommended as the synchronizing DFFs will gate the clock for a few clocks during which time the oscillator will begin to stabilize.

Limitations:

NOTE!!! This block requires at least three D-Type Flip-Flops with asynchronous reset capabilities. Typically these cells are placed at the top level of the design and instantiated by hand. The user should then manually verify the timing and placement of these cells. See the HDL source files for more details.

9.7 VSPI Serial Peripheral Interface (800 Gates)

The VSPI peripheral provides an interface to the popular SPI bus found on many microcontrollers and peripheral ICs. Typically the VSPI functions as an SPI master but can also function as a slave under software control. The timing of the SPI control signals are software programmable allowing easy interfaces to virtually all types of SPI devices. Transfer rates on the SPI bus are typically about 1 MHz and are limited to 1/8th of the clock speed of the V8.

The SPI bus is a 3 wire bus that in effect links a serial shift register between the "master" and the "slave". Typically both the master and slave have an 8 bit shift register so the combined register is 16 bits. When an SPI transfer takes place, the master and slave shift their shift registers 8 bits and thus exchange their 8 bit register values.

The VPSI core is completely software configurable. The clock polarity, clock phase, the clock frequency in master mode, and the number of bits to be transferred are all software programmable. These configuration bits are usually determined by the capabilities of the other device you wish to communicate with.

SPI supports multiple slaves on a single 3 wire bus by using separate SLave SElect signals (SVLSEL) to enable the desired slave. Multiple masters are also supported and some support is provided for detecting collisions when multiple masters attempt to transfer at the same time.

A Wired-OR mode is provided which allows multiple masters to collide on the bus without risk of damage. In this mode, an external pullup resistor is required on the SI and SO pins. WOR mode also allows the SPI bus to operate as a 2 wire bus by connecting the SI and SO pins together to form a single bidirectional data pin.

Generally, pullups are recommended on all of the external SPI signals to insure they are held in a valid state even when the VSPI core is disabled.

Limitations:

When operating as a slave, the SPI clock signal (SCK) must be slower than 1/8th of the CPU clock. 1/16th is recommended. Note that this core is fully synchronous to the CLK signal and thus SCK is sampled and then operated on. This results in 3 to 4 clocks of delay which will violate the SPI spec if SCK is faster than 1/8th of the CPU clock. When the VSPI core is in master mode, it operates exactly on the proper edges since it is generating SCK.

Register Definitions:

ADDR	NAME	R/W	DESCRIPTION
0	DOUT	W	8 Bit Data Out Register
0	DIN	R	8 Bit Data In Register
1	CTL	R/W	Control Register [0]=Reserved Always write zero [1]=MSTENB Enable SPI master mode [2]=WOR Wire-OR mode enabled [3]=CKPOL Clock Polarity 1=SCK idles high, 0=SCK idles low [4]=PHASE Phase Select [6:5]=DVD Clock divide:00=8,01=16,10=32,11=64 [7]=IRQENB Interrupt enable

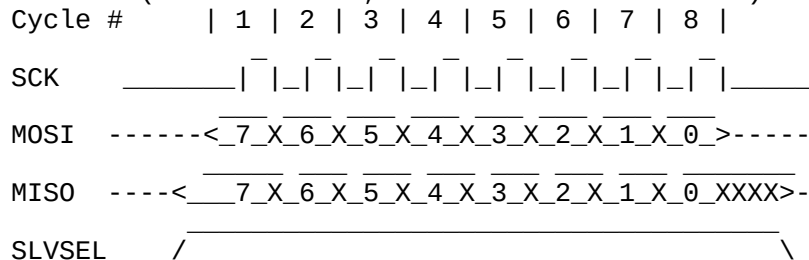
ADDR	NAME	R/W	DESCRIPTION
2	STATUS	R/W	Interrupt Status register Each bit of the status register is cleared to zero by writing ONE to the respective bit. [7]=IRQ Interrupt active Set at the end of a master mode transfer, or when SLVSEL goes high on a slave transfer [6]=Overrun This bit is set when the DOUT register is written while an SPI transfer is in progress. [5]=COL This bit is set when there is a master mode collision between multiple SPI masters. It is set when SLVSEL goes low while MSTENB=1. [2:4]=zero [1]=TXRUN (read only) 1=Master mode operation underway. [0]=SLVSEL (read only) This bit corresponds to the SLVSEL pin on the VSPI core (note that this is normally inverted at the IO pin).
3	SSEL	R/W	Slave Select/bit count register [7:5]=BITCNT Number of bits to shift in master mode, 000=8 bits, 001=1 bit, 010=2 bits, ... 111=7 bits. [4:0]=SSEL 5 individual Slave Selects for master mode.

The VSPI core operates in two fundamentally different modes based on the PHASE bit (CTL[4]). The two modes are depicted in the timing diagrams below. The key difference centers around the fact that SPI data is clocked out on one edge of the clock, and sampled on the other. The two modes select where the opposite edge DFF is placed. When PHASE=0, a negative edge flop is inserted into the shift_in path. When PHASE=1, the negative edge flop is inserted into the shift_out path to hold the data for an extra 1/2 clock.

The polarity of the clock that the VSPI shift register runs on is programmable with the CKPOL bit in the CTL register. The WOR bit configures the SPI bus to operate in an Open-Drain fashion typically used to prevent conflicts on the SPI bus when there are multiple bus masters.

Timing diagram:

PHASE=0 (POLCK=0 shown, invert SCKI if POLCK=1)



The shift register runs on the second edge of SCKI. A negative edge flop is placed in the shift_in path to sample data on the first edge of SCKI.

PHASE=1 (POLCK=0 shown, invert SCKI if POLCK=1)

Cycle #	1	2	3	4	5	6	7	8											
SCK	_____								_____										
MOSI	-----	<	7	X	6	X	5	X	4	X	3	X	2	X	1	X	0	>	-----
MISO	----	XXXXX	7	X	6	X	5	X	4	X	3	X	2	X	1	X	0	----	>
SLVSEL	___/																	_	

The shift register runs on the second edge of SCKI. A negative edge flop is placed in the shift_out path to hold data data for an extra 1/2 clock.

9.8 V8_DEICE

The V8_DEICE core is an integrated JTAG DEbugger/In-Circuit-Emulator for the V8 CPU. V8_DEICE integrates an IEEE 1149.1 JTAG Test Access Port (TAP) or “slave” controller which is typically connected to a JTAG debugger such as an IPS with the VJTAGMAS core enabled. The V8_DEICE core contains several different architectures which provide various levels of debug functionality with a corresponding increase in gate count.

The lowest gate count architecture is the EMPTY architecture. As the name implies, this architecture requires zero gates and simply connects the ports through the core.

The BASIC architecture requires approximately 3500 gates and provides sufficient logic to stop, single-step, set a single breakpoint, and read or write to memory. This level of functionality is small enough to be included in FPGAs or early ASIC runs. Typically this architecture is replaced with the EMPTY architecture when the ASIC has entered full volume production and the software is completely stable.

9.8.1 BASIC architecture features (3500 gates)

Generally, users of the V8 will simply rely on VAutomations JTAG debugger to understand and control the V8_DEICE core. The documentation provided below is provided for advanced users who wish to understand and extend the BASIC architecture features.

The BASIC architecture provides the logic to stop, single step, read/write memory, set a single breakpoint and interrogate the state of the V8 CPU. The V8_DEICE controls the V8 CPU via JTAG instructions and the scan chains described below.

Note that the V8_DEICE module uses the VJTAG submodule. The VJTAG module operates synchronously to the CLK signal on not the JTAG TCK signal. This limits the speed of the TCK signal to 1/8th of the CLK frequency and 1/16th is recommended.

ADDR Scan Chain

The ADDR scan chain consists of only the 16 bits of the V8 CPUs address bus. The address bus is setup as a separate chain to allow rapid polling of the address bus during real time operation for generating histograms. The chain is 16 bits long with bit 0 shifted out first and bit 15 shifted out last.

DATA Scan Chain

The DATA scan chain consists of the 8 bit DATAIN and DATAOUT busses. These 16 bit are appended to the end of the ADDR scan chain. The DATAIN bus is on bits 15 to 8 and the DATAOUT bus is on bits 7 to 0. The ADDR scan chain is on bits 31 to 16.

CTL Scan Chain

The CTL scan chain consists of the various control signals to and from the V8. The following table describes each bit. Note that the DATA and ADDR scan chains are appended to the CTL scan chain.

Bit Position	Signal
55:40	ADDR[15:0]
39:32	DATAIN[7:0]
31:24	DATAOUT[7:0]
23	SPARE1
22	SPARE2
21	SPARE3
20	RESET
19	READY
18	WRITE
17	READ
16	OP_FTCH
15:12	CLR_P[3:0]
11:8	SET_P[3:0]
7:0	INT[7:0]

BREAKPOINT scan chain

The BREAKPOINT scan chain is additional chain with the same bit positions as the full ADDR/DATA/CTL scan chain listed in the table above. Each bit of the BREAKPOINT scan chain is actually a mask bit. To enable a breakpoint, the desired breakpoint condition is scanned into the ADDR/DATA/CTL scan chain. Any bits in the chain that are to be part of the active condition are set to one in the BREAKPOINT scan chain. Then the BREAKPT_ENB instruction should be executed and the target processor is then allowed to run by issuing a RUN instruction. The processor will be stopped by deasserting READY **two** clock cycles after the breakpoint condition has been recognized. Note that the breakpoint cannot stop on the actual breakpoint condition due to pipeline restrictions.

STATUS scan chain

The STATUS scan chain consists of a single 8 bit register with some status information as listed in the table below.

Bit #	Description
7:2	Reserved (0)
1	Breakpoint flag This bit is a one when the breakpoint is currently being searched for. It is set when the ENB_BKPT instruction is loaded into the instruction register. It is cleared when the breakpoint has been found.
0	JTAG HALT This bit is a one when the target CPU is halted.

The INSTRUCTION register selects certain modes of operation and scan chains as described in the following table. Note that all instructions have the least significant two bits to be 01 which the INST register must be set to according to the 1149.1 specification.

INST	HEX	SCAN CHAIN	DESCRIPTION
EXTEST	00		Required by IEEE 1149.1
STOP	11	ADDR	Forces READY low. V8 stops execution immediately. READY remains low until the RUN instruction is executed.
RUN	21	ADDR	Releases READY. V8 executes normally.
ENB_BKPT	31	STATUS	Run until the breakpoint is reached
STATUS	41	STATUS	Select the STATUS scan chain
ONE_OP	51	ADDR	Release READY for one instruction. V8 advances to the next opcode fetch.
ONE_CLK	61	ADDR	Release READY for one clock. V8 executes one clock.
OP_ADDR	71	ADDR	Capture the address of the next opcode fetch.
ADDR	81	ADDR	Select the ADDR scan chain
DATA	91	DATA	Select the ADDR_DATA scan chain
CTL	A1	CTL	Select the ADDR_DATA_CTL scan chain
BOUND	B1	BOUND	Select the external BOUNDARY scan chain
BREAKPT	C1	BREAKPT	Select the BREAKPOINT scan chain
MEM_WRITE	05	DATA	Initiates a MEMORY WRITE cycle
MEM_READ	15	DATA	Initiates a MEMORY READ cycle
APPLY	45	CTL	Apply the scanned in vector for 1 clock.
APPLY_V8	55	CTL	Apply the scanned in vector for 1 clock only to the V8. Hold READ and WRITE inactive to external logic.
APPLY_EXT	65	CTL	Apply the scanned in vector for 1 clock only to external logic. Hold READY to the V8 inactive.
BYPASS	FF		BYPASS instruction required by IEEE1149.1. The BYPASS opcode is automatically loaded into the INSTRUCTION register when RESET is asserted.

10 Synthesizing the V8-uRISC

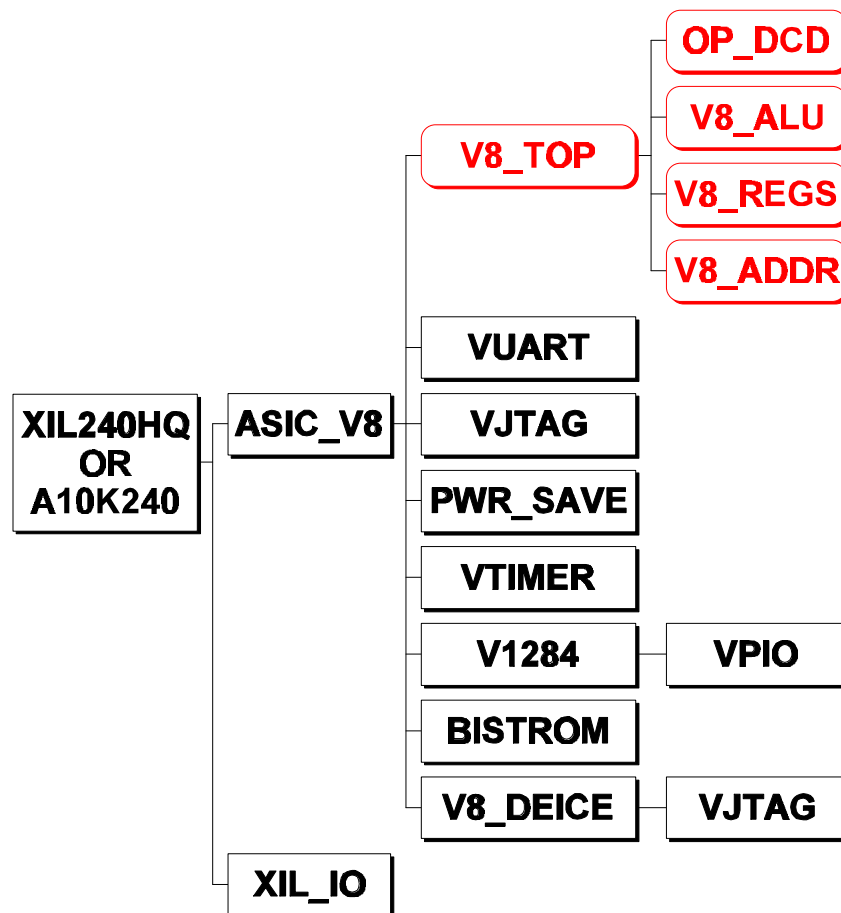
The V8-uRISC is technology, simulator and synthesizer independent. Virtually any VHDL or Verilog simulator can be used with the V8 and the test bench. Any synthesizer with sufficient capacity can synthesize the V8 due to careful HDL coding styles used by VAutomation. Synthesis scripts are provided for Synopsys Design Compiler (v8_top.dc) and for the Exemplar Leonardo FPGA synthesizer. Any target technology from ASICs to FPGAs can be used to implement the V8 in silicon as long as synthesizer libraries are available.

10.1 Makefile

The Makefile is a UNIX makefile which can be used and easily modified to compile or synthesize the model. File dependencies are included so that when analyzing for simulation or synthesizing, only the blocks that have changed will be reanalyzed or resynthesized. The Makefile supports various synthesizers and simulators and may require some environment variables to be setup in order for it to operate properly. See the comments at the top of the file for more information. For PC users, VAutomation highly recommends the MKS Toolkit from Mortice and Kern Associates [HTTP://www.mks.com](http://www.mks.com) which will provide the Unix shell commands required for the Makefile.

10.2 V8-uRISC Microcontroller File Hierarchy

The highlighted files in the drawing below indicate the files associated with the V8-uRISC CPU core. The other files are the various peripherals and other logic used to create the V8 based microcontroller used in the IntelliCore Prototype System.



File Name	Description
XIL240HQ or A10K240	Top level structural logic that creates the input, output and tristate IO cells. Clock buffering cells and the external RAM write enable logic is included here as it often requires special placement and timing constraints.
XIL_IO	Several IO cells for Xilinx and Altera FPGAs which are used to select the proper clock buffers and IO cells.
ASIC_V8	Top Level mostly structural module which interconnects all of the V8 peripherals to the V8. Some logic is created at this level for various control registers, readback muxes and address decoding.
V8_TOP	Top level module for the V8-uRISC CPU
OP_DCD	V8 Opcode decoding logic
V8_ALU	V8 Arithmetic Logic Unit
V8_REGS	16x8 dual port register file. Two architectures are provided, SYNTH and XILINX. SYNTH will result in DFF/MUX implementation of about 1700 gates, XILINX is ideal for Xilinx FPGAs and results in 8 CLBs instead of over 64.
V8_ADDR	Address generation logic
VUART	8 bit UART with integrated BAUD rate generator
V8_DEICE	JTAG debugger logic
VJTAG	IEEE 1149.1 JTAG TAP controller
VTIMER	16 Bit counter/timer
V1284	IEEE 1284 Parallel Printer port
VPIO	8 bit Programmable Parallel IO
PWR_SAVE	Power Save Unit
BISTROM	512x8 ROM behavioral model

11 Test Bench Description

A test bench is supplied along with the V8-uRISC synthesizable modules. The test bench is provided in source code format. The test bench is a system level simulation environment that emulates the IntelliCore Prototyping System.

The key to using the test bench effectively is to build a complete system-level simulation environment of your system. A simulation environment enables you to debug your ASIC before you have committed the design to silicon. You can also integrate and debug software in the HDL simulator and make hardware/software design tradeoffs. The test bench provided is an example of the IntelliCore Prototype System (IPS) which is available from VAutomation.

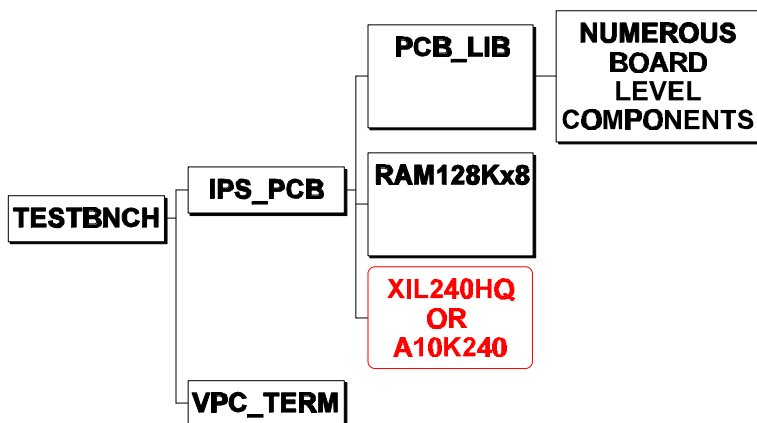
There are two primary files used for controlling the test bench, the VPC_TERM.DAT and RAM128k8.HEX files. The VPC_TERM.DAT file simulates commands entered from a terminal connected to the RS-232 port of the IPS. With this port you can enter commands just as you would in the real IPS. Also, any debugging information that would normally be transmitted out the RS-232 port is echoed to the screen during simulation.

The RAM128k8.HEX file is the Intel Hex format file from the V8ASM assembler. This is how you get your programs into the simulator for the V8 to execute. Note the ROMs for Xilinx FPGAs are built using the ROMBLDR program which is supplied and also available on VAutomations web page.

11.1 V8-uRISC Test Bench File Hierarchy

There are numerous files associated with the V8-uRISC test bench. The important files are shown in the diagram below. The TESTBNCH module is the top level and simply contains a small amount of structural logic to interconnect the VPC_TERM module to the IPS_PCB module. In VHDL the TESTBNCH module also includes a couple of architectures which can be used to simulate different configurations of the V8. Typically you will want to select the A_V8_BY_ITSELF configuration.

The TESTBNCH module in turn instantiates the IPS_PCB which is a structural model that was used to build the IntelliCore IPS printed circuit board. The IPS_PCB module in turn calls out many small modules that have been placed in the PCB_LIB subdirectory for simplicity sake. All of the modules in the PCB_LIB subdirectory are behavioral and are only used for simulation purposes. Most of the modules are actually empty and are needed as place holders to match the components on the PCB.



The IPS_PCB module instantiates the XIL240HQ and A10K240 modules which are the FPGA versions of the V8 describer in the previous section. The important files are described briefly below. See the headers in the files themselves for more detail.

File Name	Description
TESTBNCH	Top level structural module with configurations in VHDL.
VPC_TERM	Reads the VPC_TERM.DAT file and transmits the commands into the V8-uRISC RS-232 port. Observes the RS-232 transmit channel and echoes any characters to the simulator transcript window.
IPS_PCB	Structural model of the Top PCB of the IntelliCore Prototype System
RAM128K8	128Kx8 behavioral model that can read Intel hex files output by the V8ASM assembler. Input filename is RAM128K8.HEX.

11.2 Executing the Test Bench

Running a simulation of the V8 Test Bench is obviously simulator dependent. Below is a brief description of how to run the test bench on the Model Technology simulator. The figure below shows a timing diagram of the test bench simulation using the Model Technology simulator.

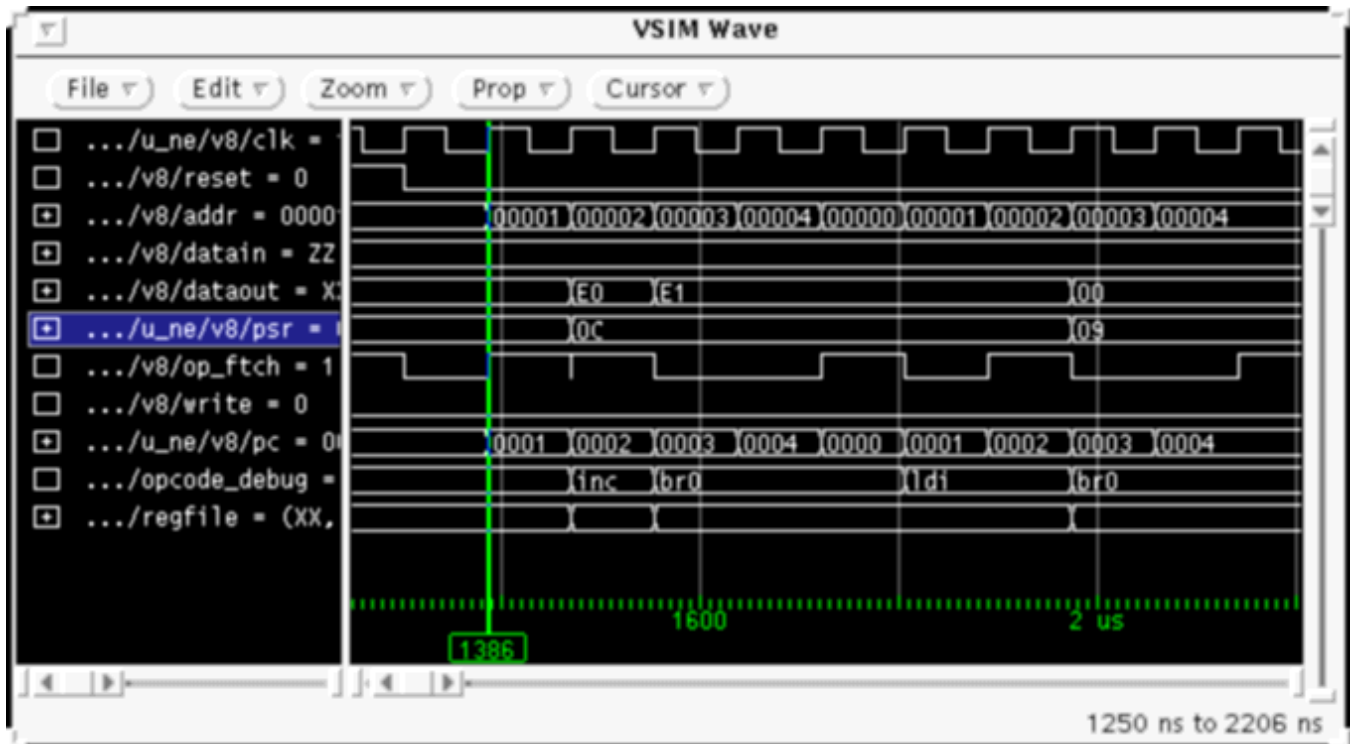


Figure 6

Two different sets of software are built when the Makefile is run. One set is for use in the simulator and the other is for use in actual hardware such as the IPS. In the simulator, we don't want to wait for long loops to finish such as clearing all of RAM or operating the UART at the actual baud rate of 19.2K. Instead, the WANT_HARDWARE symbol in the V8 assembly routines can significantly shorten runtimes by skipping over many of these time consuming items. Generally, the files for downloading to hardware end in `_hw` (IE: `bistromhw.hex` is used to build a xilinx FPGA prototype but the `bistrom.hex` file is used in simulation).

To compile the design:
make sim

We have provided a sample STARTUP.DO file which will provide some additional controls and automatically bring up a number of useful signals in the WAVE window. You may need to modify some parameters in your MODELSIM.INI file to get the simulation to work.

To enter the simulator:
Click on the Vsystem icon or type
vsim

Select the following configuration to load into the simulator
Design Name: a_v8_in_asic or a_v8_in_fpga

Vsystem should load the design
Do startup.do

should run the simulator long enough for the regression test to complete. A message should be printed to the transcript window indicating that the regression test has completed. Note that the simulation does not stop automatically. It will run forever polling for characters to be entered from a terminal!

11.3 Notes for Verilog Users

VAutomation develops all cores in VHDL and translates the code into Verilog. While the translators are 100% effective for all of the synthesizable code, VHDL provides better FileIO capabilities than Verilog. Thus there are some limitations in the Verilog test bench. Specifically, the VPC_TERM.DAT file is converted to a hex file with the Unix command line:

```
od -hvw1 vpc_term.dat 0x0 | sed -e 's/^0000/@/' -e 's/00$/!' >! vpc_term.hex
```

Any changes to the VPC_TERM.DAT file must be retranslated by executing the above command.

The command line above is executed whenever the bistrom file is rebuilt. Be sure to type
make hex
before entering Cadence Verilog-XL.

Verilog also does not support the concept of different architectures for the same module. To provide the same functionality available in VHDL, separate files are provided for the modules that have multiple architectures. Many of the peripherals have EMPTY architectures that make it easy to remove a peripheral simply by reading in the VUART_EMPTY.V file instead of the VUART.V file.

For an example on invoking the Cadence Verilog-XL simulator, see the DEMO.RUN file.

11.4 Assembly language software

The V8-uRISC core comes with some diagnostic regression test assembly source code. The assembly source code is provided in the SOFTWARE subdirectory. Use the source code as an example of how to code your application.

Brief descriptions of the files are listed below. See the headers of the files for more detail.

File Name	Description
REGRESSS and REGRESSH	These two files (S for simulation and H for download to hardware) are built from a C program using the HiTech C compiler. This is the main regression test normally run in the simulations.
V8_DEMO	Top level assembly source code for the demonstration application.
AD1848	Audio Chip initialization and drivers
BIST	128 byte Built-in-Self test.
BISTROM	Combines the BIST and BOOT programs
BOOT	384 byte Boot code. Reads data from serial I2C EEPROMs and copies it into SRAM. If the checksum of the EEPROMs fails, then it will wait and download new code from the UART.
CONFIGV8	Various IO mappings and equates
FW_DRV	Placeholder for the 1394 drivers
I2C	I2C serial EEPROM bus drivers
REGRESS	V8 self-checking Regression tests
UART	UART drivers

12 V8 Software Development Tools

The V8 comes with an Assembler and a software simulator which are described in detail in the **V8 Software Development Tools** manual (v8tools.pdf). In addition, a JTAG debugger allows debug of silicon implementations of the V8 to be debugged using an interface that is nearly identical to the V8 software simulator. A full ANSI C compiler is available from HiTech software [HTTP://WWW.htsoft.com](http://www.htsoft.com).

13 Tips and Techniques

13.1 Debugging V8 software in an HDL simulator

Simulations of the V8 in an HDL simulator can provide significant time-to-market advantages by enabling hardware-software co-design, debug and integration. The first step in this process is to build a detailed system-level simulation environment. The V8 ships with a fairly complete environment for the IntelliCore Prototype System which can be easily modified to match your own environment.

Once the simulation environment is ready, software can be loaded via an Intel Hex file into the RAMs or ROMs in the system and debug can begin in the HDL simulator. This approach has been used quite extensively by VAutomation in debugging the drivers for the VUSB Core. One advantage of the HDL simulator is that you have full access to all nodes in the design at all times. This allows quick and easy debug of state machines or other hardware designs. However, when debugging software, the large amount of data presented by the simulator can make following program flow much like finding a needle in a haystack. One trick we have found to quickly locate software routines which are currently being debugged is to add a STP (4-7) opcode to the start of a routine and a CLP opcode to the end. This technique makes it very easy to find the routine being tested in the simulator waveform viewer by simply displaying the PSR register bits individually. You can then quickly zoom in on regions which are suspect and complete the debug process by viewing the address and data busses as well as the internal registers. Remember that the upper 4 bits of the PSR are basically under software control and you can think of these bits as single bit flags. Note that viewing bit 3 of the PSR makes it very obvious when the V8 is processing interrupts and can be used to calculate how much overhead is available in the various interrupt service routines.

13.2 Interfacing to Off-Chip Memories

When interfacing the V8 to off-chip memories such as SRAM or FLASH devices, you may have difficulties meeting the address and data hold times relative to the write enable. To solve this problem, gate WRITE with the clock to create a clean WE_N which meets hold time of the SRAM. Gating WRITE with the clock is safe because WRITE comes directly from a flop. The clock will force WR_N to the SRAM high during the first half of the clock cycle and drives WR_N high with plenty of hold time available for address and data. See the XIL240HQ module for an example of this as implemented in the IntelliCore Prototype System.

```
WR_N <= NOT (WRITE AND NOT CLK);
```

13.3 Expanding the Address Space Beyond 64K bytes

The V8 directly addresses a 64K byte address space. For applications requiring more than 64K bytes of RAM, a page register is added to provide typically 8 Megabytes of off-chip RAM which is windowed in the upper 32K byte space. This window to off-chip RAM can be of any practical size. Typically code space is traded off for a larger window into the off-chip RAM. Thus, if your code can easily fit in the low 32K, then a large 32K off-chip RAM window is ideal. Alternatively, if you need nearly the full 64K for code, then the off-chip RAM window would be as small as practical (typically 1K bytes). This technique is implemented in the V8_MCU module.

Using a page register is a classic technique for increasing the addressing capabilities of 8 bit microprocessors. As an example, we can increase the V8s addressing capabilities to 8 Megabytes by adding an 8 bit page register. The page register is typically mapped to one of the hardware register address such as 0000. This register is then used to provide address bits 23 through 15 to the off-chip RAM. Address bits 14 through 0 come directly from the V8 address bus. Address bit 15 of the V8 is used to enable the data from off chip RAM. **Figure 7** is a diagram of a circuit that can be used to provide off-chip RAM addressing.

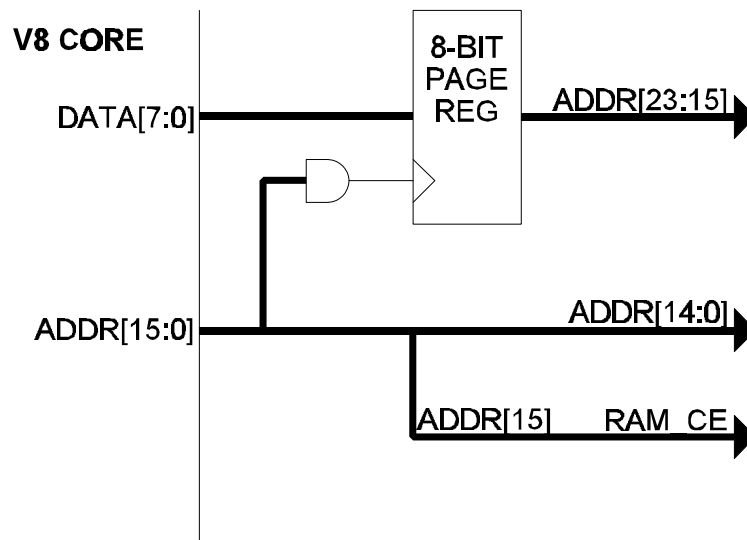


Figure 7 Page Register Example

14 IntelliCore™ Prototype System

A complete FPGA prototyping system for the V8 is available from VAutomation. Details on the IntelliCore™ Prototype System can be found on VAutomation's Internet Web site at [HTTP://www.vautomation.com/v8/ips.html](http://www.vautomation.com/v8/ips.html).